ОТЧЁТ

по лабораторной работе № 3
по дисциплине «Программирование и администрирование
в среде системы управления базами данных»

«Учёт квартплаты: сальдо, начисления и платежи по лицевым счетам»

Выполнил(а): Скоромникова Юлия
Группа: 15.27Д-МО03/23б

Цель и задание работы

Цель работы: научиться проектировать базу данных для учёта операций по лицевым счетам, создавать веб-интерфейс для заполнения таблиц и оформлять отчёты (сводки) в виде хранимых процедур MySQL.

Задание:

1. Создать базу данных MySQL.
2. Создать в базе данных три таблицы: saldo (сальдо по лицевому счёту), charges (начисления по лицевому счёту), payments (платежи по лицевому счёту).
3. Реализовать WEB-интерфейс для заполнения таблиц.
4. Реализовать стандартные модули вывода информации на основе хранимых процедур (3 модуля): оборотная ведомость (в клетке за месяц первая строка – начисления по квартире, вторая – платёж, третья – сальдо на конец месяца), оборотная ведомость по квартире, сводка по категориям должников.

Сальдо – бухгалтерский термин, обозначающий итог на начало или конец периода по счёту. В работе принято: положительное сальдо означает долг жильца, отрицательное – переплату (аванс). Сальдо на конец месяца вычисляется по формуле:

сальдо на конец = сальдо на начало + начислено − оплачено.

Среда выполнения

Работа выполнялась на стенде, развёрнутом с помощью Docker Compose (файл docker-compose.yml) и состоящем из трёх контейнеров:

– **db** – СУБД MySQL 8.0, кодировка utf8mb4, сравнение utf8mb4_unicode_ci (порт 3308);

– **phpmyadmin** – веб-интерфейс администрирования phpMyAdmin (http://localhost:8081);

– **web** – веб-сервер Apache с PHP 8.3 и расширением pdo_mysql, на котором работает интерфейс учёта квартплаты (http://localhost:8001).

При первом запуске контейнер db автоматически выполняет сценарии из каталога sql: 01_schema.sql (база данных и таблицы), 02_data.sql (тестовые данные) и 03_procedures.sql (хранимые процедуры).

1. Создание базы данных и таблиц

Создана база данных kvartplata с кодировкой utf8mb4, что позволяет хранить русские тексты (фамилии жильцов, названия услуг).

```
CREATE DATABASE IF NOT EXISTS kvartplata
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE kvartplata;
```

Кроме трёх таблиц из задания создан справочник лицевых счетов accounts. На него ссылаются таблицы saldo, charges и payments, поэтому номер квартиры, фамилия квартиросъёмщика и площадь хранятся в одном месте и не повторяются в каждой записи. Связь обеспечивается внешними ключами (FOREIGN KEY) с каскадным удалением.

Для контроля правильности данных используются ограничения:

– UNIQUE – нельзя дважды ввести сальдо на один и тот же месяц или начисление за одну услугу в одном месяце;

– CHECK – месяц хранится как дата 1-го числа, номер лицевого счёта состоит из 6–10 цифр, площадь и сумма платежа положительны;

– FOREIGN KEY – запись нельзя привязать к несуществующему лицевому счёту.

Суммы хранятся в типе DECIMAL(12,2), чтобы денежные расчёты выполнялись без ошибок округления, свойственных типам FLOAT и DOUBLE.

1.1. Таблица accounts (лицевые счета)

Таблица 1 – Реквизитный состав таблицы accounts

Поле	Тип	Описание
id	INT UNSIGNED, PK	идентификатор
account_no	VARCHAR(10), UNIQUE	номер лицевого счёта (6–10 цифр)
flat	SMALLINT UNSIGNED, UNIQUE	номер квартиры
owner	VARCHAR(150)	ответственный квартиросъёмщик
area	DECIMAL(6,2)	общая площадь, м²
residents	TINYINT UNSIGNED	число проживающих

```
CREATE TABLE accounts (
    id         INT UNSIGNED  NOT NULL AUTO_INCREMENT,
    account_no VARCHAR(10)   NOT NULL COMMENT 'Номер лицевого счёта',
    flat       SMALLINT UNSIGNED NOT NULL COMMENT 'Номер квартиры',
    owner      VARCHAR(150)  NOT NULL COMMENT 'Ответственный квартиросъёмщик',
    area       DECIMAL(6,2)  NOT NULL COMMENT 'Общая площадь, м²',
    residents  TINYINT UNSIGNED NOT NULL DEFAULT 1 COMMENT 'Число проживающих',
    PRIMARY KEY (id),
    UNIQUE KEY uq_account_no (account_no),
    UNIQUE KEY uq_flat (flat),
    CONSTRAINT chk_account_no CHECK (account_no REGEXP '^[0-9]{6,10}$'),
    CONSTRAINT chk_area CHECK (area > 0)
) ENGINE = InnoDB COMMENT = 'Лицевые счета';
```

1.2. Таблица saldo (сальдо по лицевому счёту)

Хранит сальдо лицевого счёта на начало месяца. Обязательно вводится входящее сальдо на начало года; сальдо следующих месяцев либо вычисляется процедурами, либо записывается при закрытии месяца (процедура sp_close_month, раздел 3.5).

Таблица 2 – Реквизитный состав таблицы saldo

Поле	Тип	Описание
id	INT UNSIGNED, PK	идентификатор
account_id	INT UNSIGNED, FK → accounts	лицевой счёт
period	DATE (1-е число)	месяц, на начало которого зафиксировано сальдо
amount	DECIMAL(12,2)	сумма сальдо: «+» долг, «−» переплата

```
CREATE TABLE saldo (
    id         INT UNSIGNED  NOT NULL AUTO_INCREMENT,
    account_id INT UNSIGNED  NOT NULL COMMENT 'Лицевой счёт',
    period     DATE          NOT NULL COMMENT 'Месяц (1-е число), на начало которого
зафиксировано сальдо',
    amount     DECIMAL(12,2) NOT NULL COMMENT 'Сальдо: + долг / − переплата',
    PRIMARY KEY (id),
    UNIQUE KEY uq_saldo (account_id, period),
    CONSTRAINT fk_saldo_account FOREIGN KEY (account_id) REFERENCES accounts (id)
        ON DELETE CASCADE,
    CONSTRAINT chk_saldo_period CHECK (DAYOFMONTH(period) = 1)
) ENGINE = InnoDB COMMENT = 'Сальдо по лицевому счёту';
```

1.3. Таблица charges (начисления по лицевому счёту)

Начисления хранятся в разрезе услуг: содержание жилья, отопление, холодная и горячая вода, водоотведение, электроэнергия, обращение с ТКО, капитальный ремонт. Отрицательная сумма означает перерасчёт (уменьшение начисления).

Таблица 3 – Реквизитный состав таблицы charges

Поле	Тип	Описание
id	INT UNSIGNED, PK	идентификатор
account_id	INT UNSIGNED, FK → accounts	лицевой счёт
period	DATE (1-е число)	расчётный месяц
service	VARCHAR(50)	услуга
amount	DECIMAL(12,2)	сумма начисления

```
CREATE TABLE charges (
    id         INT UNSIGNED  NOT NULL AUTO_INCREMENT,
    account_id INT UNSIGNED  NOT NULL COMMENT 'Лицевой счёт',
    period     DATE          NOT NULL COMMENT 'Расчётный месяц (1-е число)',
    service    VARCHAR(50)   NOT NULL COMMENT 'Услуга',
    amount     DECIMAL(12,2) NOT NULL COMMENT 'Сумма начисления (отрицательная — 
перерасчёт)',
    PRIMARY KEY (id),
    UNIQUE KEY uq_charge (account_id, period, service),
    KEY ix_charges_period (period),
    CONSTRAINT fk_charges_account FOREIGN KEY (account_id) REFERENCES accounts (id)
        ON DELETE CASCADE,
    CONSTRAINT chk_charges_period CHECK (DAYOFMONTH(period) = 1)
) ENGINE = InnoDB COMMENT = 'Начисления по лицевому счёту';
```

1.4. Таблица payments (платежи по лицевому счёту)

Платёж учитывается в том месяце, в котором деньги поступили на расчётный счёт (поле pay_date).

Таблица 4 – Реквизитный состав таблицы payments

Поле	Тип	Описание
id	INT UNSIGNED, PK	идентификатор
account_id	INT UNSIGNED, FK → accounts	лицевой счёт
pay_date	DATE	дата поступления на расчётный счёт
amount	DECIMAL(12,2), > 0	сумма платежа
source	VARCHAR(30)	способ оплаты: банк, онлайн, касса, почта
doc_no	VARCHAR(20), NULL	номер платёжного документа

```
CREATE TABLE payments (
    id         INT UNSIGNED  NOT NULL AUTO_INCREMENT,
    account_id INT UNSIGNED  NOT NULL COMMENT 'Лицевой счёт',
    pay_date   DATE          NOT NULL COMMENT 'Дата поступления на расчётный счёт',
    amount     DECIMAL(12,2) NOT NULL COMMENT 'Сумма платежа',
    source     VARCHAR(30)   NOT NULL DEFAULT 'Банк' COMMENT 'Способ оплаты',
    doc_no     VARCHAR(20)   NULL COMMENT 'Номер платёжного документа',
    PRIMARY KEY (id),
    KEY ix_payments_account_date (account_id, pay_date),
    CONSTRAINT fk_payments_account FOREIGN KEY (account_id) REFERENCES accounts (id)
        ON DELETE CASCADE,
    CONSTRAINT chk_payments_amount CHECK (amount > 0)
) ENGINE = InnoDB COMMENT = 'Платежи по лицевому счёту';
```

Список таблиц базы данных в phpMyAdmin показан на рисунке 1, структура таблицы charges с индексами – на рисунке 2, внешний ключ – на рисунке 3.

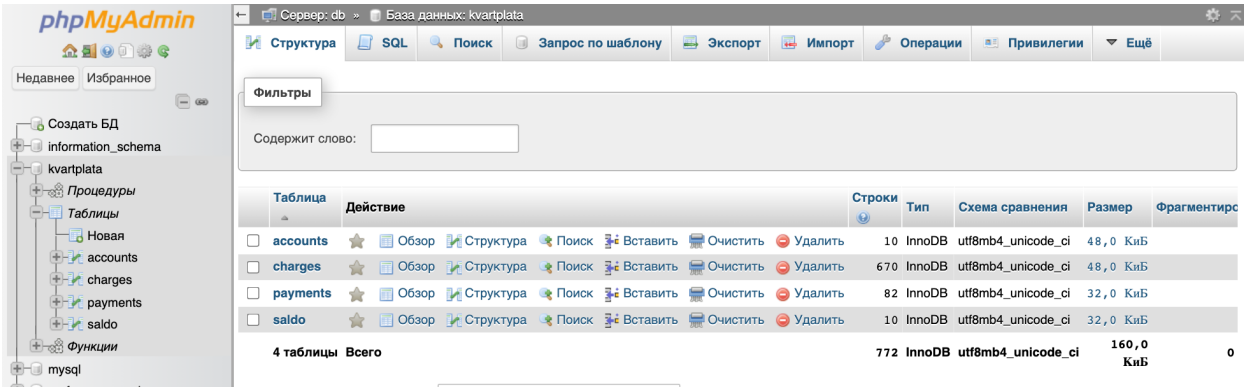


Рисунок 1 – Таблицы базы данных kvartplata в phpMyAdmin

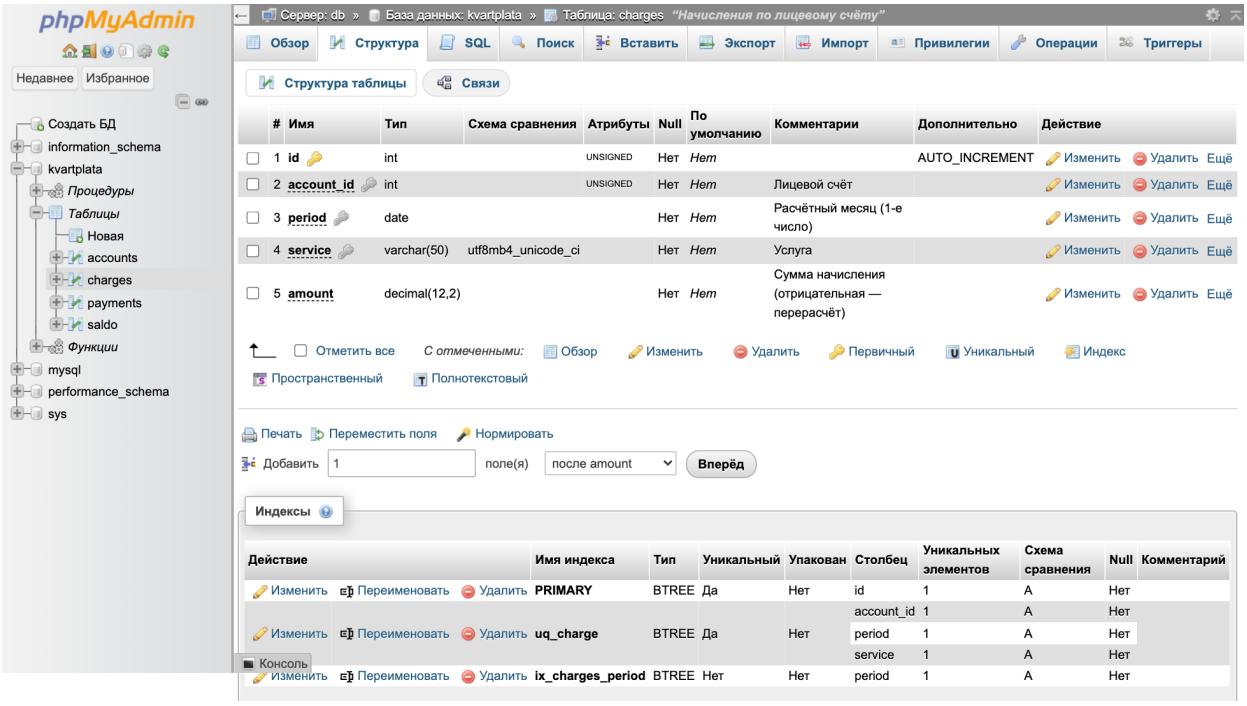


Рисунок 2 – Структура таблицы charges в phpMyAdmin

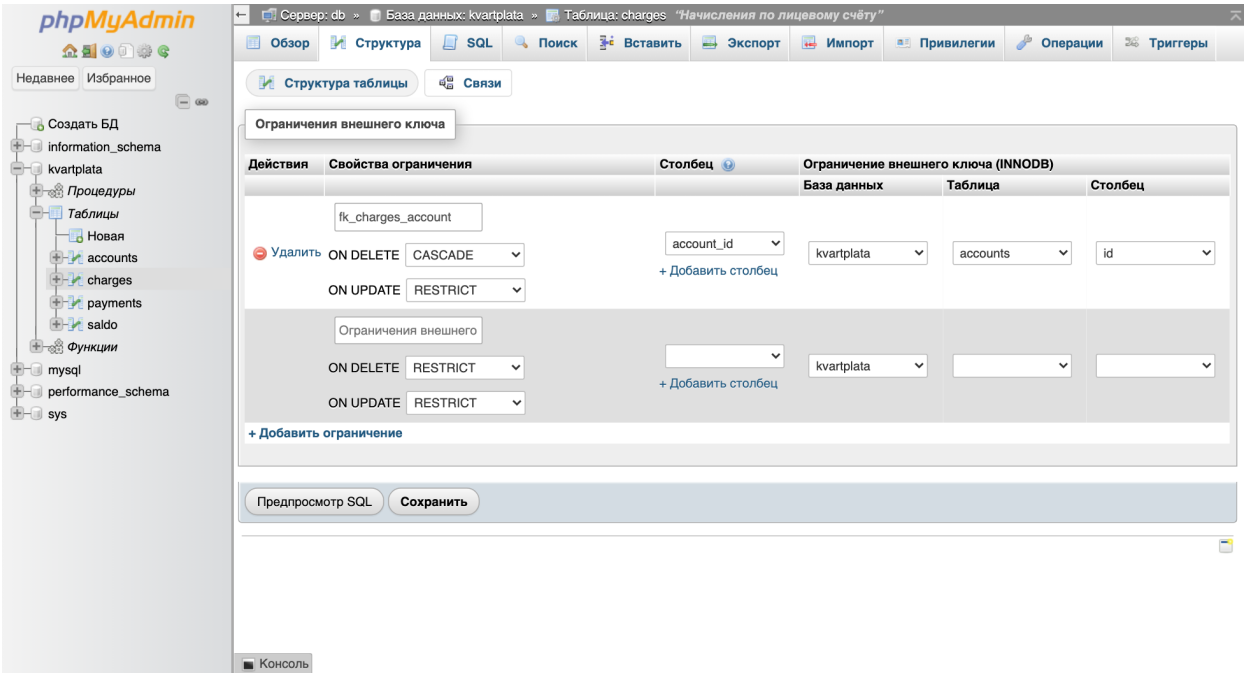


Рисунок 3 – Внешний ключ fk_charges_account таблицы charges

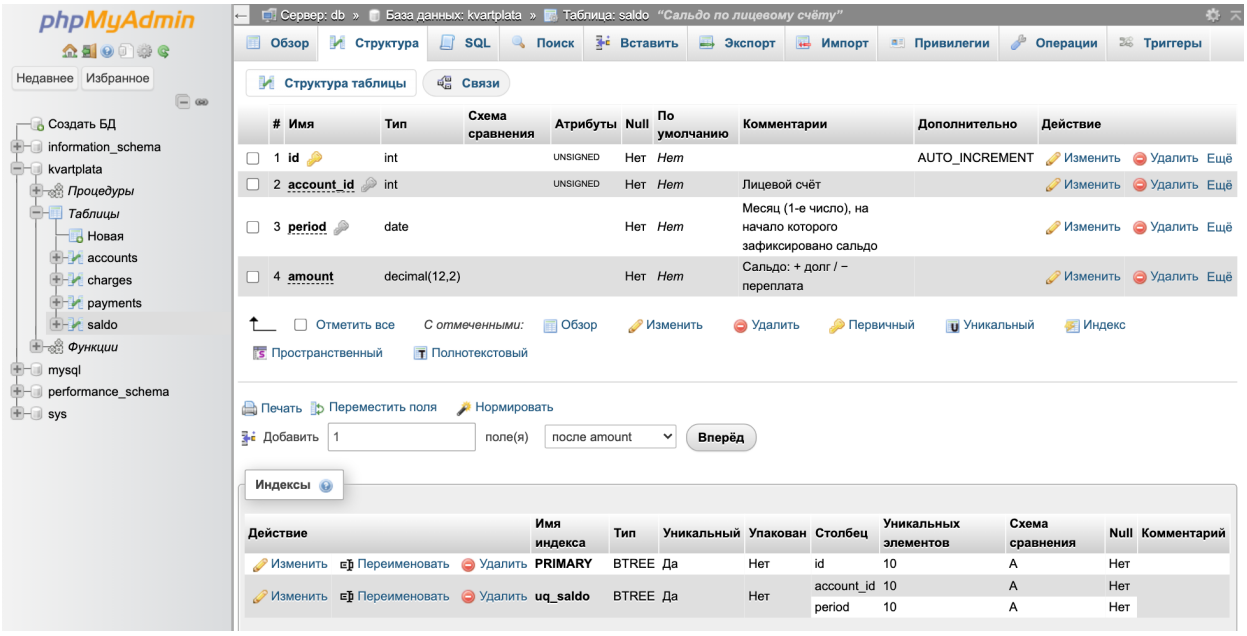


Рисунок 4 – Структура таблицы saldo в phpMyAdmin

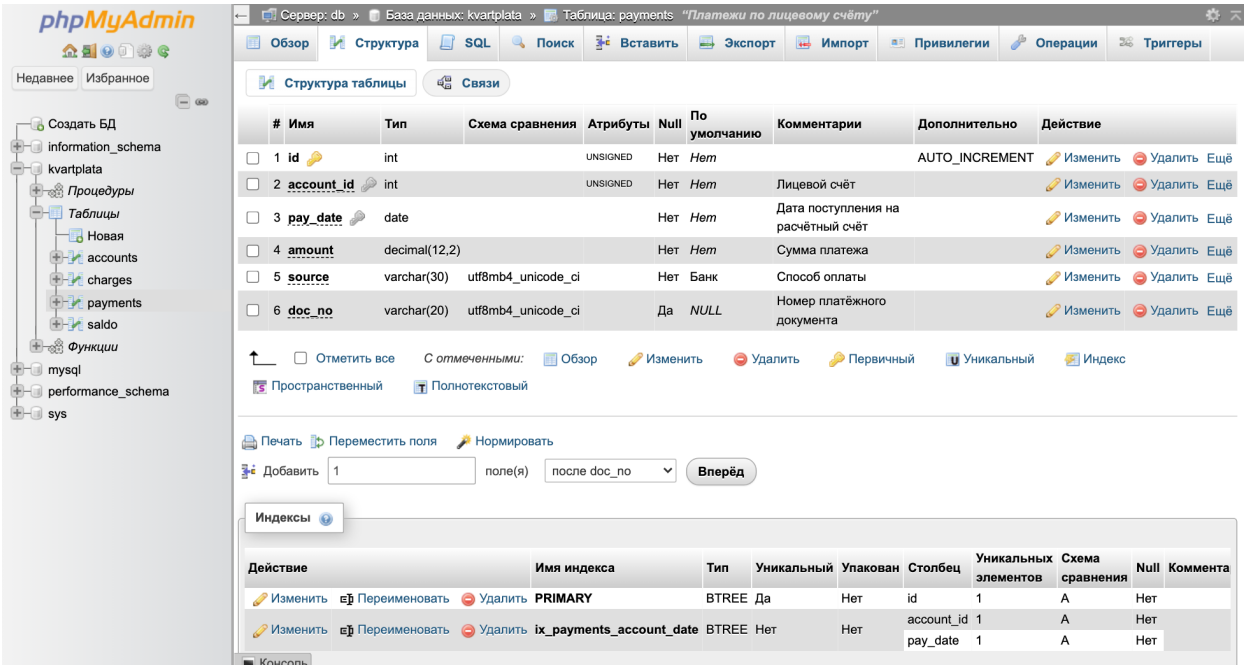


Рисунок 5 – Структура таблицы payments в phpMyAdmin

1.5. Тестовые данные

Для проверки отчётов таблицы заполнены данными по 10 квартирам за январь–сентябрь 2026 года (файл sql/02_data.sql): входящее сальдо на 01.01.2026, 670 начислений по восьми услугам и 82 платежа. Данные подобраны так, чтобы в отчётах встречались разные ситуации: жильцы, которые платят каждый месяц; должники на 1, 2, 3 и более месяцев; квартира 6 с переплатой (платит авансом раз в два месяца).

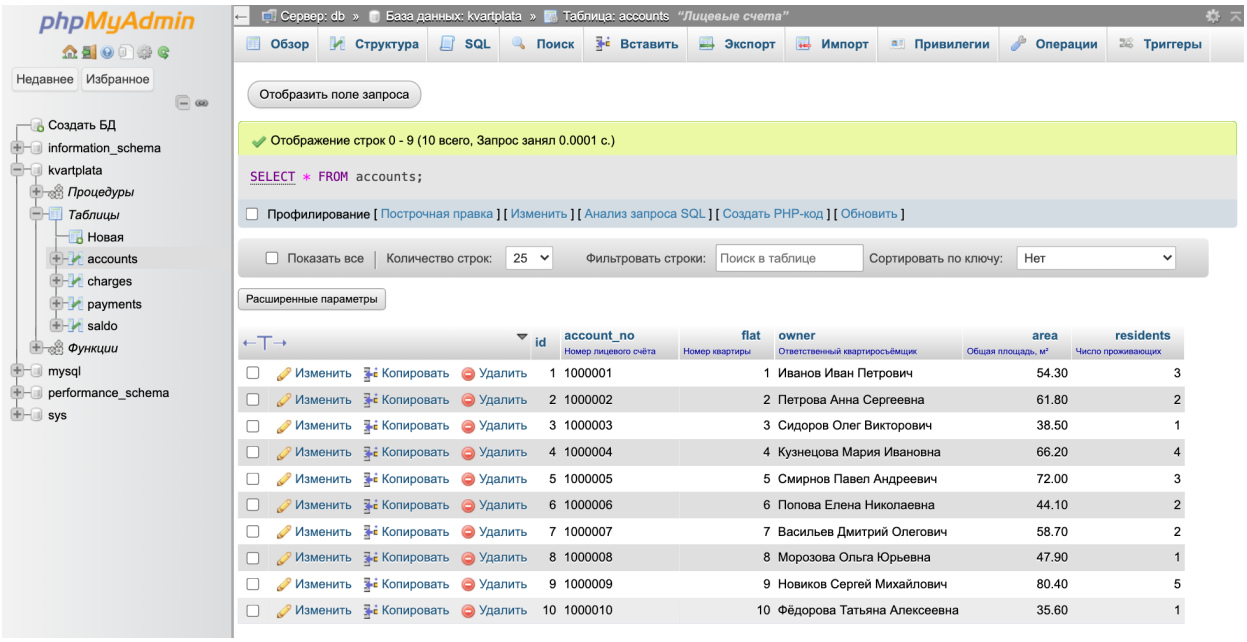


Рисунок 6 – Содержимое таблицы accounts

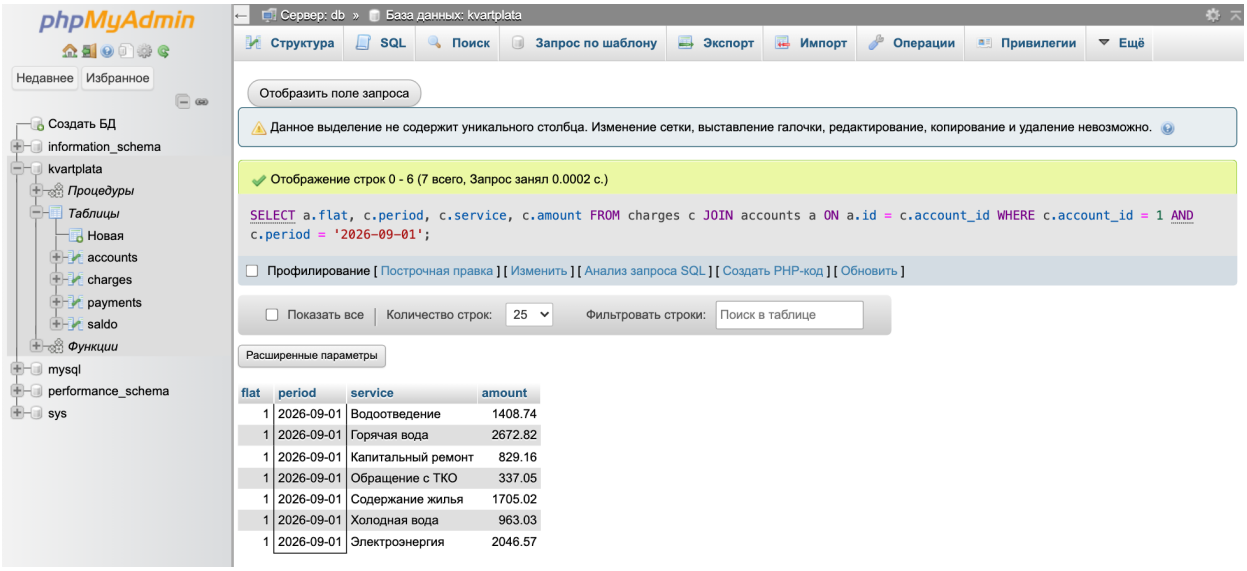


Рисунок 7 – Начисления по квартире 1 за сентябрь 2026 года

2. WEB-интерфейс для заполнения таблиц

Интерфейс реализован в виде набора веб-страниц на PHP (каталог web). Страницы подключаются к базе данных через PDO. Все значения из форм передаются в запросы через подготовленные запросы (prepared statements), что защищает от SQL-инъекций.

Таблица 5 – Файлы веб-интерфейса

Файл	**Назначение**
lib.php	подключение к БД, вызов процедур, общая шапка и меню
index.php	главная страница: итоговые показатели и ссылки на разделы
edit.php	ввод, изменение и удаление записей во всех четырёх таблицах
turnover.php	модуль 1 – оборотная ведомость
account.php	модуль 2 – оборотная ведомость по квартире
debtors.php	модуль 3 – сводка по категориям должников
style.css	оформление страниц

Возможности интерфейса заполнения таблиц:

– добавление, изменение и удаление записей в таблицах accounts, saldo, charges и payments;
– выбор квартиры из выпадающего списка, ввод месяца и даты через календарь;
– фильтр списка записей по квартире и по месяцу, строка «Итого» с суммой;
– проверка обязательных полей и понятные сообщения об ошибках (повторная запись, нарушение ограничения CHECK, удаление используемой записи);
– закрытие месяца – запись рассчитанного сальдо на начало следующего месяца;
– цветовое выделение сальдо: долг – красным, переплата – зелёным.

Все четыре таблицы обслуживает одна страница edit.php. Состав полей каждой таблицы описан в массиве $tables, а форма ввода, список записей и SQL-запросы строятся по этому описанию. Фрагмент кода, сохраняющий запись:

```
$id = (int)($_POST['id'] ?? 0);
$cols = array_keys($values);
if ($id) {
    $set = implode(', ', array_map(fn($c) => "$c = ?", $cols));
    db()->prepare("UPDATE $t SET $set WHERE id = ?")
        ->execute([...array_values($values), $id]);
} else {
    db()->prepare(sprintf('INSERT INTO %s (%s) VALUES (%s)', $t,
        implode(', ', $cols), implode(', ', array_fill(0, count($cols), '?'))))
        ->execute(array_values($values));
}
```

Ошибки СУБД перехватываются и переводятся в понятные сообщения по коду ошибки MySQL:

```
$error = match ($e->errorInfo[1] ?? 0) {
    1062    => 'Такая запись уже существует (нарушена уникальность)',
    3819    => 'Значение не прошло проверку (CHECK): ' . $e->getMessage(),
    1451    => 'Запись используется в других таблицах',
    default => $e->getMessage(),
};
```

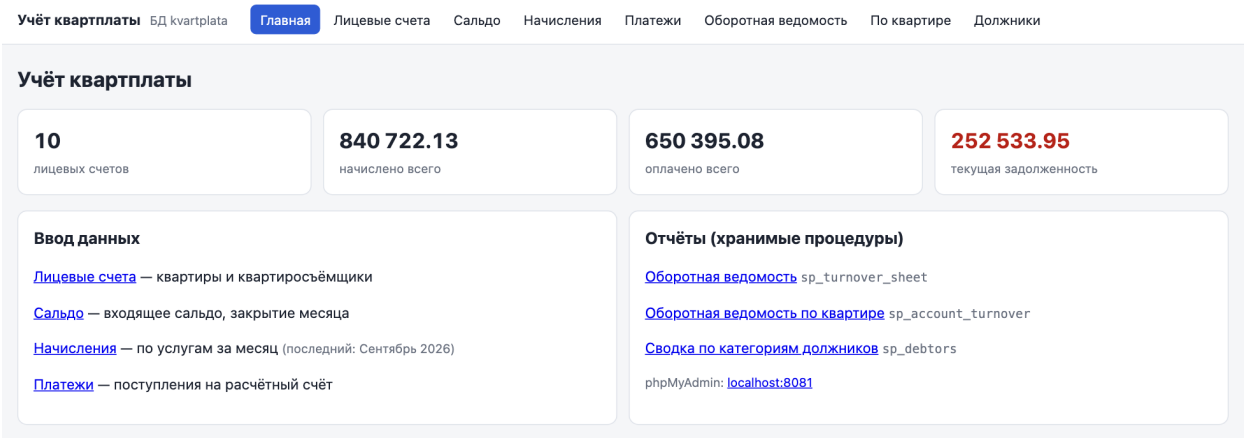


Рисунок 8 – Главная страница веб-интерфейса

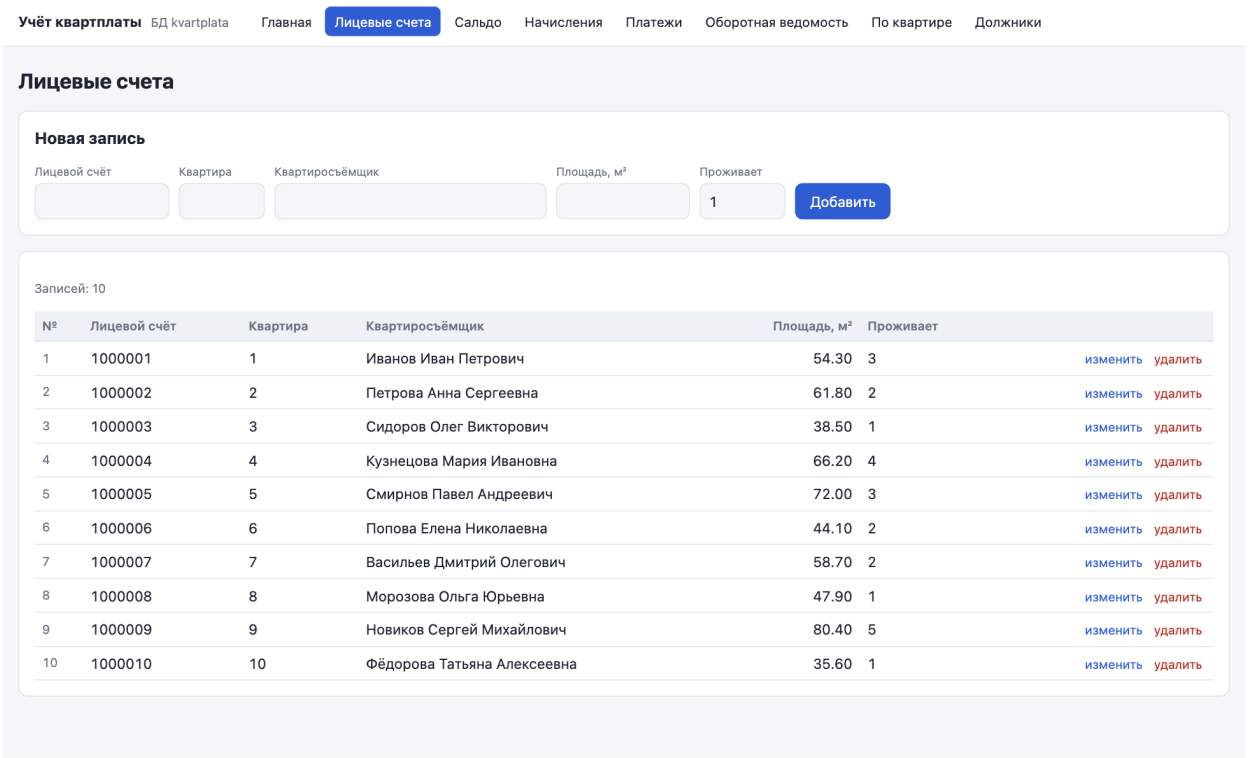


Рисунок 9 – Ввод и просмотр лицевых счетов

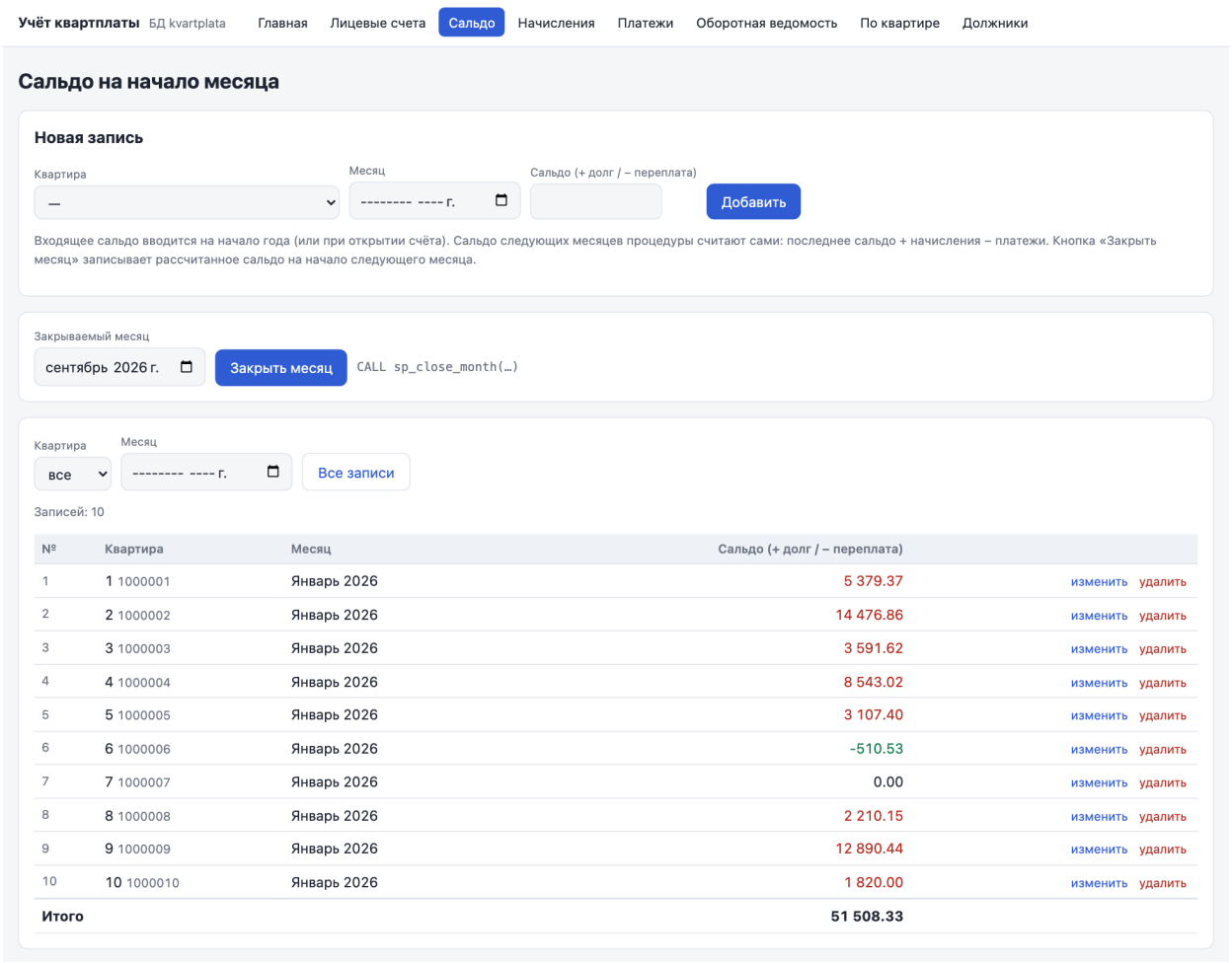


Рисунок 10 – Ввод входящего сальдо и закрытие месяца

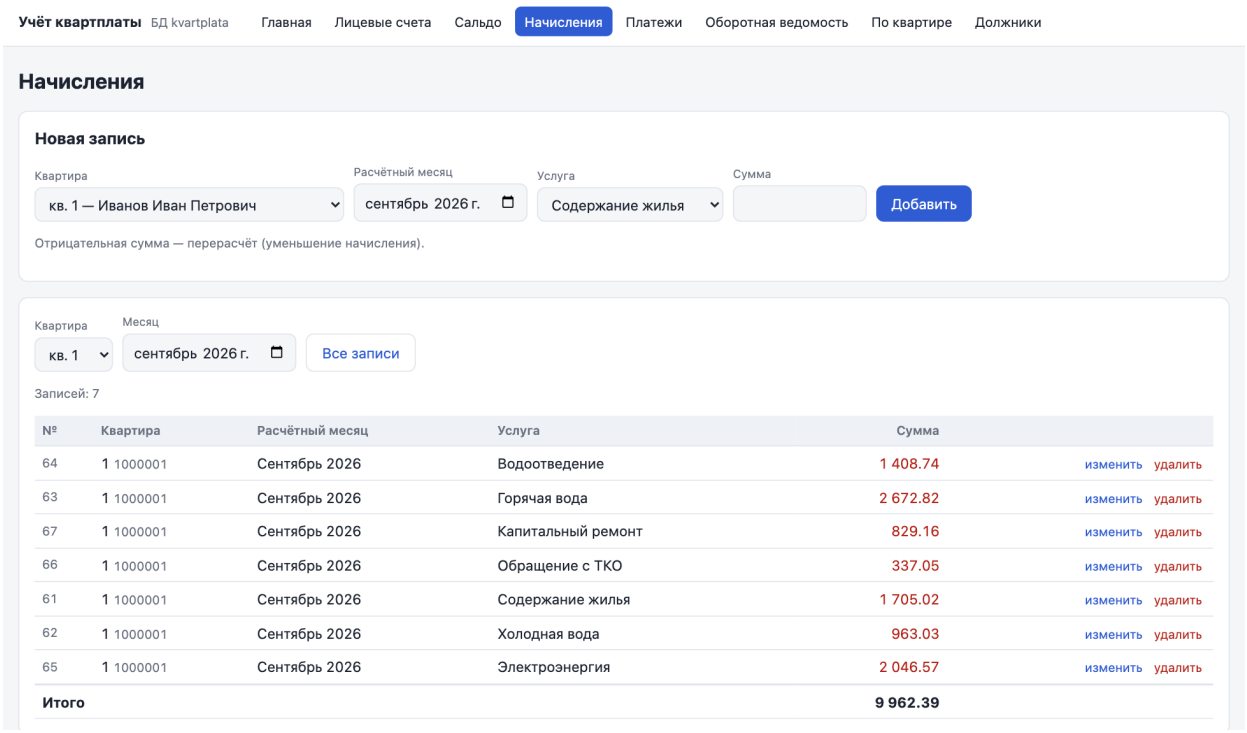


Рисунок 11 – Ввод начислений: фильтр по квартире 1 за сентябрь 2026 года

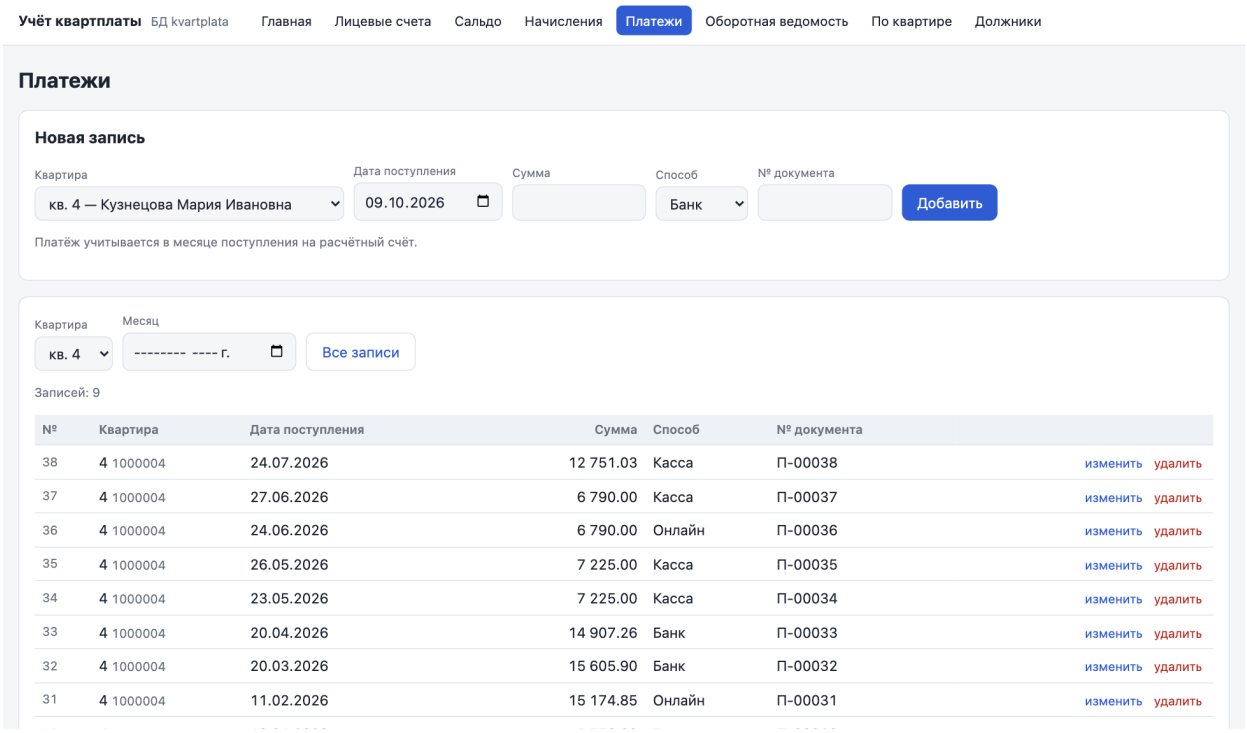


Рисунок 12 – Ввод платежей: фильтр по квартире 4

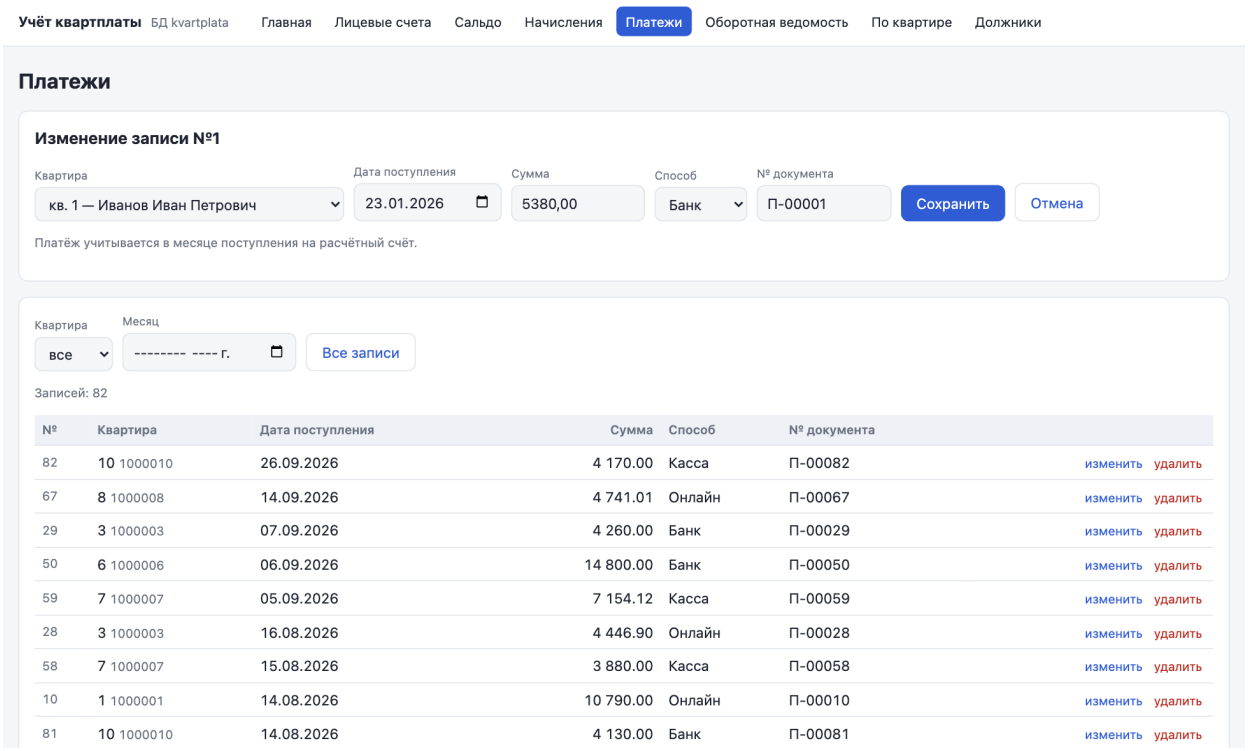


Рисунок 13 – Изменение существующего платежа

При попытке повторно ввести начисление за услугу «Содержание жилья» по квартире 1 за сентябрь 2026 года срабатывает уникальный ключ uq_charge, и интерфейс выводит сообщение об ошибке (рисунок 14).

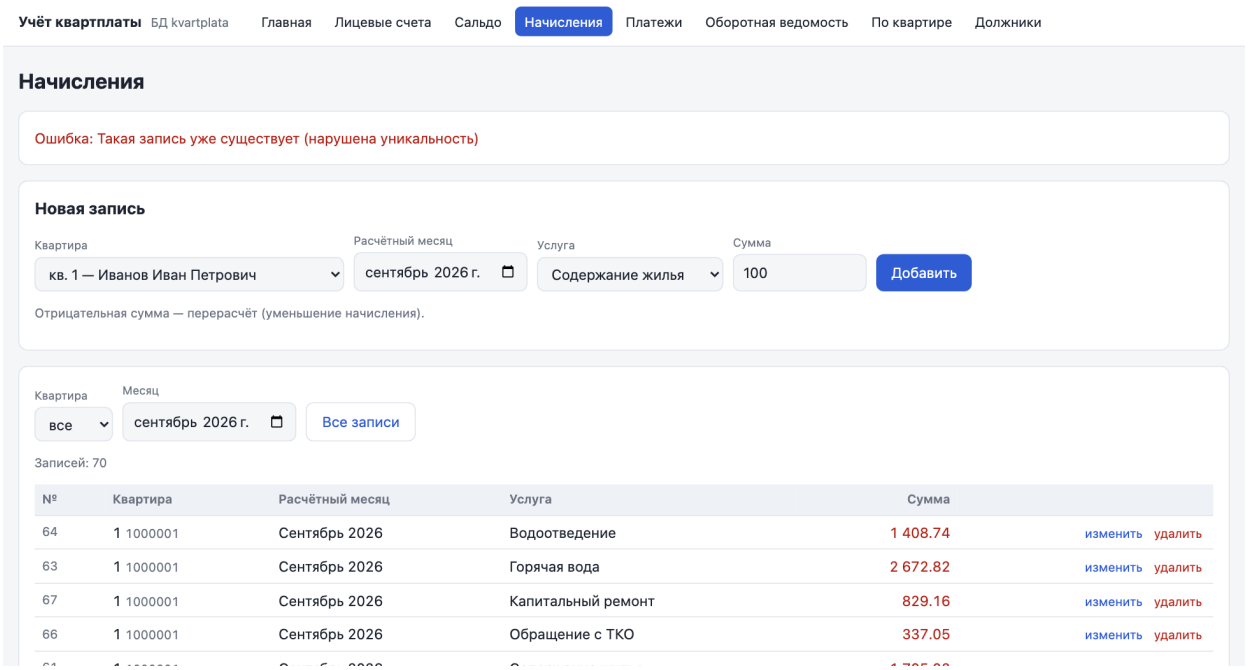


Рисунок 14 – Сообщение об ошибке при вводе повторного начисления

3. Модули вывода информации на основе хранимых процедур

Хранимая процедура – это набор SQL-операторов, который хранится на сервере базы данных под определённым именем и вызывается командой CALL. Все расчёты отчётов (суммирование начислений и платежей, вычисление сальдо, отнесение долга к категории) выполняются на сервере, а веб-страницы только вызывают процедуру и оформляют полученный результат в таблицу.

Созданные процедуры и функция перечислены в таблице 6.

Таблица 6 – Хранимые процедуры и функции базы данных kvartplata

Процедура / функция	**Назначение**
`fn_saldo(account_id, date)`	функция: сальдо лицевого счёта на начало указанной даты
`sp_turnover_sheet(year)`	модуль 1: оборотная ведомость за год по всем квартирам
`sp_account_turnover(account_id, year)`	модуль 2: оборотная ведомость по квартире (два набора результатов)
`sp_debtors(date)`	модуль 3: сводка по категориям должников на дату
`sp_close_month(period)`	закрытие месяца: запись сальдо на начало следующего месяца

Процедуры созданы сценарием sql/03_procedures.sql. Команда DELIMITER $$ меняет разделитель, чтобы точки с запятой внутри тела процедуры не завершали команду CREATE PROCEDURE. В процедурах используются общие табличные выражения (WITH), рекурсивное CTE для генерации списка месяцев и оконная функция SUM() OVER для накопительного итога, которые поддерживаются начиная с MySQL 8.0. Полный текст процедур приведён в приложении А.

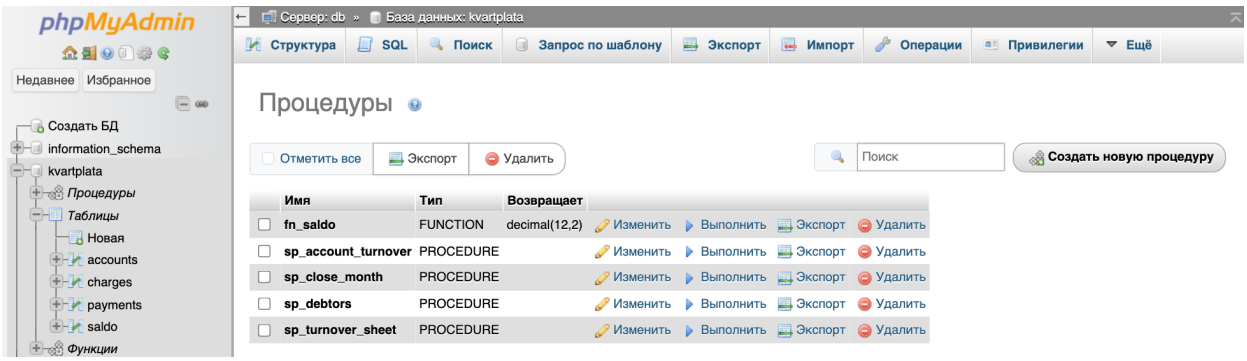


Рисунок 15 – Список хранимых процедур и функций базы данных kvartplata в phpMyAdmin

3.1. Функция fn_saldo

Функция вычисляет сальдо лицевого счёта на начало заданной даты. Она находит последнюю запись в таблице saldo не позже этой даты, прибавляет начисления и вычитает платежи, поступившие после неё. Функция используется всеми тремя модулями.

```
CREATE FUNCTION fn_saldo(p_account_id INT UNSIGNED, p_date DATE)
    RETURNS DECIMAL(12,2)
    READS SQL DATA
    COMMENT 'Сальдо лицевого счёта на начало даты'
BEGIN
    DECLARE v_from   DATE DEFAULT '1000-01-01';
    DECLARE v_amount DECIMAL(12,2) DEFAULT 0;
    DECLARE v_charged, v_paid DECIMAL(12,2);

    SELECT period, amount INTO v_from, v_amount
    FROM saldo
    WHERE account_id = p_account_id AND period <= p_date
    ORDER BY period DESC
    LIMIT 1;

    SELECT COALESCE(SUM(amount), 0) INTO v_charged
    FROM charges
    WHERE account_id = p_account_id AND period >= v_from AND period < p_date;

    SELECT COALESCE(SUM(amount), 0) INTO v_paid
    FROM payments
    WHERE account_id = p_account_id AND pay_date >= v_from AND pay_date < p_date;

    RETURN v_amount + v_charged - v_paid;
END$$
```

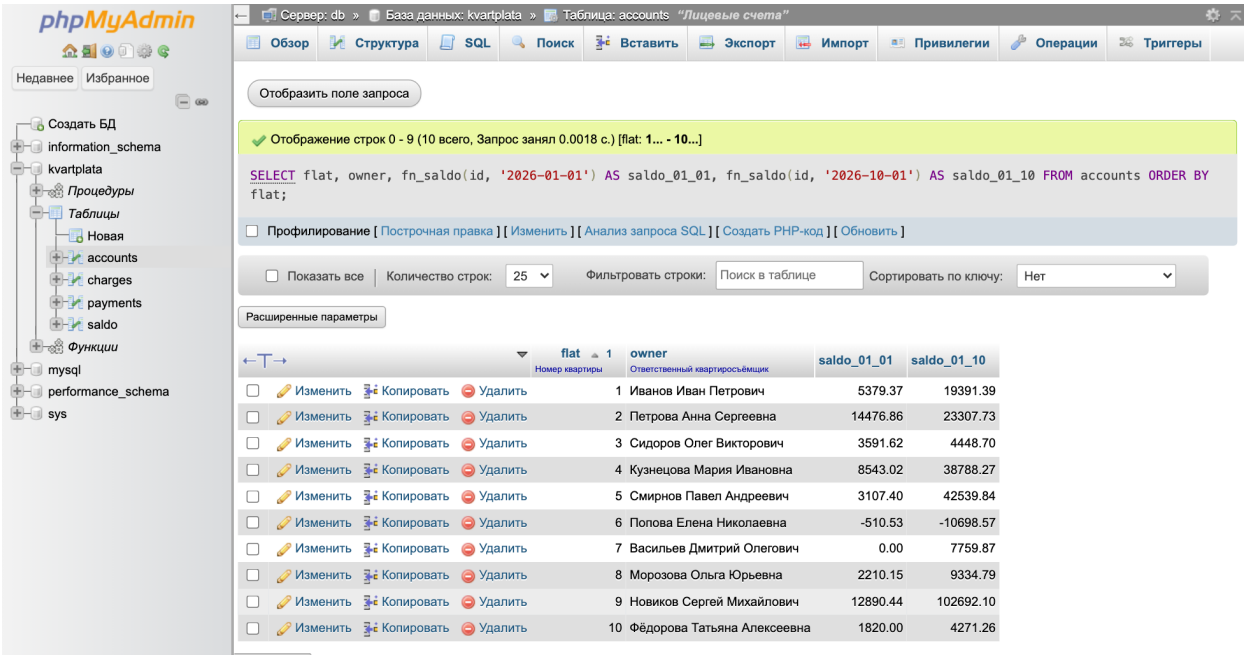


Рисунок 16 – Вызов fn_saldo: сальдо на 01.01.2026 и на 01.10.2026

3.2. Модуль 1. Оборотная ведомость

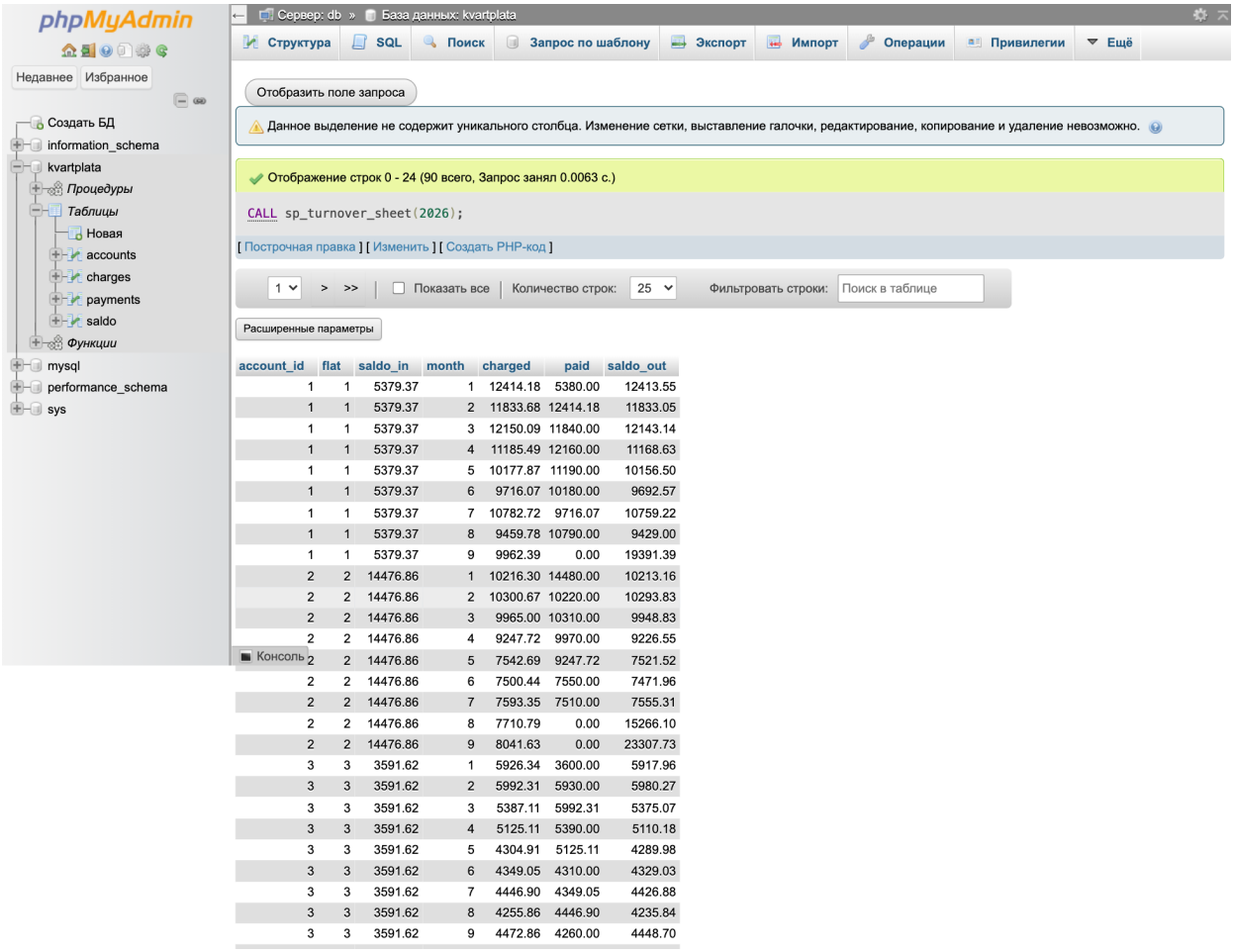
Процедура sp_turnover_sheet возвращает по каждой квартире входящее сальдо на начало года и для каждого месяца – сумму начислений, сумму платежей и сальдо на конец месяца. Список месяцев строится рекурсивным CTE months; сальдо на конец месяца вычисляется оконной функцией как накопительная сумма разности «начислено − оплачено», прибавленная к входящему сальдо.

```
CREATE PROCEDURE sp_turnover_sheet(IN p_year SMALLINT)
    COMMENT 'Оборотная ведомость за год по всем квартирам'
BEGIN
    DECLARE v_start DATE DEFAULT MAKEDATE(p_year, 1);
    DECLARE v_last  TINYINT;

    SELECT MAX(m) INTO v_last FROM (
        SELECT MONTH(period) AS m FROM charges WHERE YEAR(period) = p_year
        UNION ALL
        SELECT MONTH(pay_date) FROM payments WHERE YEAR(pay_date) = p_year
    ) t;

    WITH RECURSIVE months AS (
        SELECT 1 AS m
        UNION ALL
        SELECT m + 1 FROM months WHERE m < COALESCE(v_last, 0)
    ),
    ch AS (
        SELECT account_id, MONTH(period) AS m, SUM(amount) AS amount
        FROM charges WHERE YEAR(period) = p_year
        GROUP BY account_id, MONTH(period)
    ),
    pay AS (
        SELECT account_id, MONTH(pay_date) AS m, SUM(amount) AS amount
        FROM payments WHERE YEAR(pay_date) = p_year
        GROUP BY account_id, MONTH(pay_date)
    ),
```

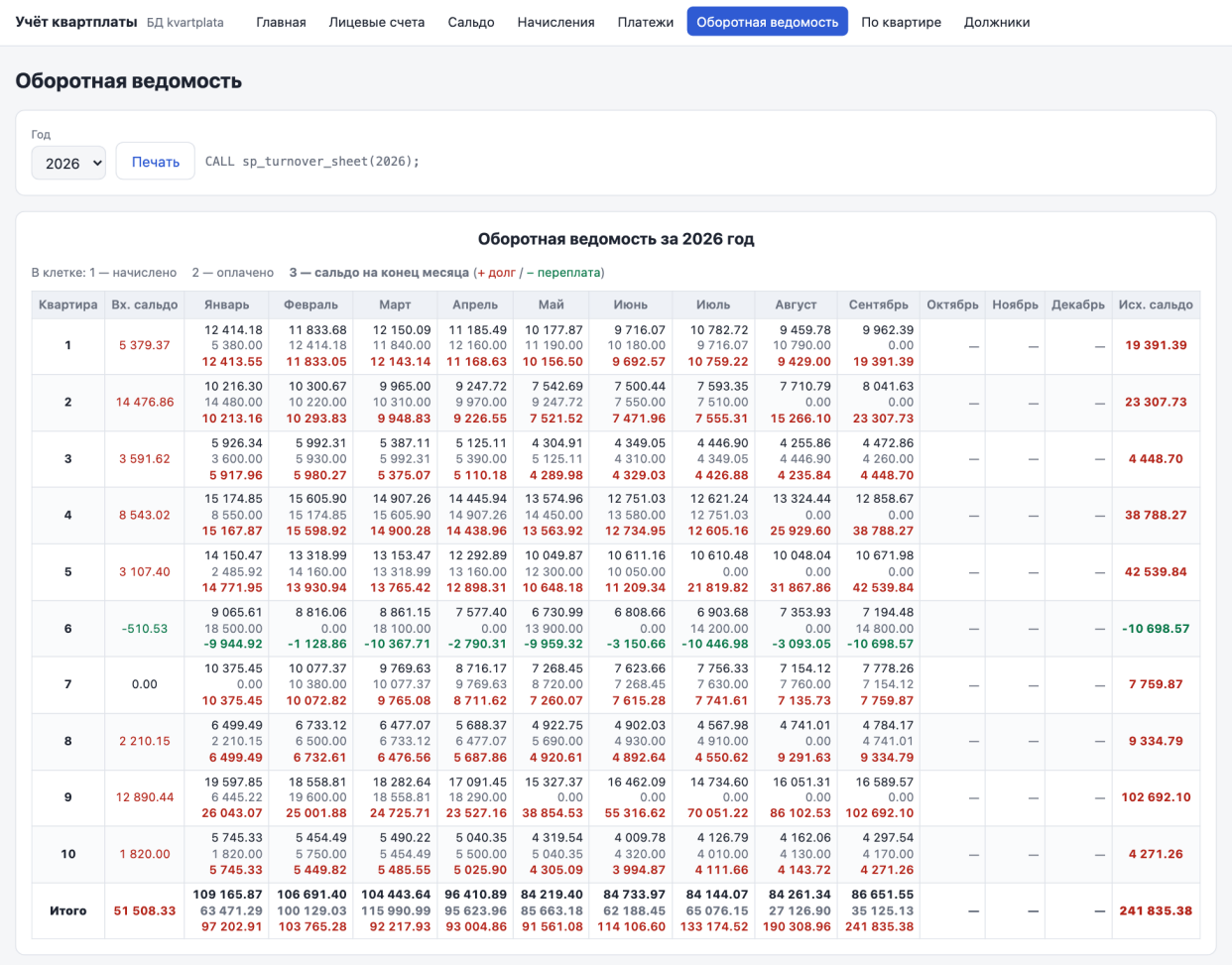
```
    grid AS (
        SELECT a.id AS account_id, a.flat, fn_saldo(a.id, v_start) AS saldo_in,
               mo.m, COALESCE(ch.amount, 0) AS charged, COALESCE(pay.amount, 0) AS paid
        FROM accounts a
        CROSS JOIN months mo
        LEFT JOIN ch  ON ch.account_id  = a.id AND ch.m  = mo.m
        LEFT JOIN pay ON pay.account_id = a.id AND pay.m = mo.m
    )
    SELECT account_id, flat, saldo_in, m AS month, charged, paid,
           saldo_in + SUM(charged - paid) OVER (PARTITION BY account_id ORDER BY m) AS
saldo_out
    FROM grid
    ORDER BY flat, m;
END$$
```



account_id	flat	saldo_in	month	charged	paid	saldo_out
1	1	5379.37	1	12414.18	5380.00	12413.55
1	1	5379.37	2	11833.68	12414.18	11833.05
1	1	5379.37	3	12150.09	11840.00	12143.14
1	1	5379.37	4	11185.49	12160.00	11168.63
1	1	5379.37	5	10177.87	11190.00	10156.50
1	1	5379.37	6	9716.07	10180.00	9692.57
1	1	5379.37	7	10782.72	9716.07	10759.22
1	1	5379.37	8	9459.78	10790.00	9429.00
1	1	5379.37	9	9962.39	0.00	19391.39
2	2	14476.86	1	10216.30	14480.00	10213.16
2	2	14476.86	2	10300.67	10220.00	10293.83
2	2	14476.86	3	9965.00	10310.00	9948.83
2	2	14476.86	4	9247.72	9970.00	9226.55
2	2	14476.86	5	7542.69	9247.72	7521.52
2	2	14476.86	6	7500.44	7550.00	7471.96
2	2	14476.86	7	7593.35	7510.00	7555.31
2	2	14476.86	8	7710.79	0.00	15266.10
2	2	14476.86	9	8041.63	0.00	23307.73
3	3	3591.62	1	5926.34	3600.00	5917.96
3	3	3591.62	2	5992.31	5930.00	5980.27
3	3	3591.62	3	5387.11	5992.31	5375.07
3	3	3591.62	4	5125.11	5390.00	5110.18
3	3	3591.62	5	4304.91	5125.11	4289.98
3	3	3591.62	6	4349.05	4310.00	4329.03
3	3	3591.62	7	4446.90	4349.05	4426.88
3	3	3591.62	8	4255.86	4446.90	4235.84
3	3	3591.62	9	4472.86	4260.00	4448.70

Рисунок 17 – Вызов sp_turnover_sheet(2026) в phpMyAdmin (первые строки результата)

Страница turnover.php сворачивает строки «квартира × месяц» в таблицу, как в образце задания: в клетке за месяц первая строка – начисления, вторая – платёж, третья – сальдо на конец месяца. Последняя строка содержит итоги по дому (рисунок 18).



Квартира	Вх. сальдо	Январь	Февраль	Март	Апрель	Май	Июнь	Июль	Август	Сентябрь	Октябрь	Ноябрь	Декабрь	Исх. сальдо
1	5 379.37	12 414.18 5 380.00 12 413.55	11 833.68 12 414.18 11 833.05	12 150.09 11 840.00 12 143.14	11 185.49 12 160.00 11 168.63	10 177.87 11 190.00 10 156.50	9 716.07 10 180.00 9 692.57	10 782.72 9 716.07 10 759.22	9 459.78 10 790.00 9 429.00	9 962.39 0.00 19 391.39	—	—	—	19 391.39
2	14 476.86	10 216.30 14 480.00 10 213.16	10 300.67 10 220.00 10 293.83	9 965.00 10 310.00 9 948.83	9 247.72 9 970.00 9 226.55	7 542.69 9 247.72 7 521.52	7 500.44 7 550.00 7 471.96	7 593.35 7 510.00 7 555.31	7 710.79 0.00 15 266.10	8 041.63 0.00 23 307.73	—	—	—	23 307.73
3	3 591.62	5 926.34 3 600.00 5 917.96	5 992.31 5 930.00 5 980.27	5 387.11 5 992.31 5 375.07	5 125.11 5 390.00 5 110.18	4 304.91 5 125.11 4 289.98	4 349.05 4 310.00 4 329.03	4 446.90 4 349.05 4 426.88	4 255.86 4 446.90 4 235.84	4 472.86 4 260.00 4 448.70	—	—	—	4 448.70
4	8 543.02	15 174.85 8 550.00 15 167.87	15 605.90 15 174.85 15 598.92	14 907.26 15 605.90 14 900.28	14 445.94 14 907.26 14 438.96	13 574.96 14 450.00 13 563.92	12 751.03 13 580.00 12 734.95	12 621.24 12 751.03 12 605.16	13 324.44 0.00 25 929.60	12 858.67 0.00 38 788.27	—	—	—	38 788.27
5	3 107.40	14 150.47 2 485.92 14 771.95	13 318.99 14 160.00 13 930.94	13 153.47 13 318.99 13 765.42	12 292.89 13 160.00 12 898.31	10 049.87 12 300.00 10 648.18	10 611.16 10 050.00 11 209.34	10 610.48 0.00 21 819.82	10 048.04 0.00 31 867.86	10 671.98 0.00 42 539.84	—	—	—	42 539.84
6	-510.53	9 065.61 18 500.00 -9 944.92	8 816.06 0.00 -1 128.86	8 861.15 18 100.00 -10 367.71	7 577.40 0.00 -2 790.31	6 730.99 13 900.00 -9 959.32	6 808.66 0.00 -3 150.66	6 903.68 14 200.00 -10 446.98	7 353.93 0.00 -3 093.05	7 194.48 14 800.00 -10 698.57	—	—	—	-10 698.57
7	0.00	10 375.45 0.00 10 375.45	10 077.37 10 380.00 10 072.82	9 769.63 10 077.37 9 765.08	8 716.17 9 769.63 8 711.62	7 268.45 8 720.00 7 260.07	7 623.66 7 268.45 7 615.28	7 756.33 7 630.00 7 741.61	7 154.12 7 760.00 7 135.73	7 778.26 7 154.12 7 759.87	—	—	—	7 759.87
8	2 210.15	6 499.49 2 210.15 6 499.49	6 733.12 6 500.00 6 732.61	6 477.07 6 733.12 6 476.56	5 688.37 6 477.07 5 687.86	4 922.75 5 690.00 4 920.61	4 902.03 4 930.00 4 892.64	4 567.98 4 910.00 4 550.62	4 741.01 0.00 9 291.63	4 784.17 4 741.01 9 334.79	—	—	—	9 334.79
9	12 890.44	19 597.85 6 445.22 26 043.07	18 558.81 19 600.00 25 001.88	18 282.64 18 558.81 24 725.71	17 091.45 18 290.00 23 527.16	15 327.37 0.00 38 854.53	16 462.09 0.00 55 316.62	14 734.60 0.00 70 051.22	16 051.31 0.00 86 102.53	16 589.57 0.00 102 692.10	—	—	—	102 692.10
10	1 820.00	5 745.33 1 820.00 5 745.33	5 454.49 5 750.00 5 449.82	5 490.22 5 454.49 5 485.55	5 040.35 5 500.00 5 025.90	4 319.54 5 040.35 4 305.09	4 009.78 4 320.00 3 994.87	4 126.79 4 010.00 4 111.66	4 162.06 4 130.00 4 143.72	4 297.54 4 170.00 4 271.26	—	—	—	4 271.26
Итого	51 508.33	109 165.87 63 471.29 97 202.91	106 691.40 100 129.03 103 765.28	104 443.64 115 990.99 92 217.93	96 410.89 95 623.96 93 004.86	84 219.40 85 663.18 91 561.08	84 733.97 62 188.45 114 106.60	84 144.07 65 076.15 133 174.52	84 261.34 27 126.90 190 308.96	86 651.55 35 125.13 241 835.38	—	—	—	241 835.38

Рисунок 18 – Оборотная ведомость за 2026 год в веб-интерфейсе

Проверка по квартире 1: входящее сальдо 5 379.37, в январе начислено 12 414.18 и оплачено 5 380.00, сальдо на конец января 5 379.37 + 12 414.18 − 5 380.00 = 12 413.55, что совпадает с третьей строкой клетки.

3.3. Модуль 2. Оборотная ведомость по квартире

Процедура sp_account_turnover возвращает два набора результатов: помесячные начисления, платежи и сальдо на конец месяца, а также итоговую строку – сальдо на начало года, всего начислено и оплачено, долг или переплату за предыдущие периоды и итоговую сумму к оплате. Сумма к оплате равна сальдо на конец периода; «долг за предыдущие периоды» – это сумма к оплате без начислений последнего месяца.

```
CREATE PROCEDURE sp_account_turnover(IN p_account_id INT UNSIGNED, IN p_year SMALLINT)
    COMMENT 'Начисления и платежи по месяцам для одной квартиры'
BEGIN
    DECLARE v_start    DATE DEFAULT MAKEDATE(p_year, 1);
    DECLARE v_saldo    DECIMAL(12,2) DEFAULT fn_saldo(p_account_id, v_start);
    DECLARE v_charged, v_paid, v_last_charge DECIMAL(12,2);
```

```
    WITH RECURSIVE months AS (
        SELECT 1 AS m UNION ALL SELECT m + 1 FROM months WHERE m < 12
    ),
    grid AS (
        SELECT mo.m,
               (SELECT COALESCE(SUM(amount), 0) FROM charges
                 WHERE account_id = p_account_id
                   AND period = DATE_ADD(v_start, INTERVAL mo.m - 1 MONTH)) AS charged,
               (SELECT COALESCE(SUM(amount), 0) FROM payments
                 WHERE account_id = p_account_id
                   AND YEAR(pay_date) = p_year AND MONTH(pay_date) = mo.m) AS paid
        FROM months mo
    )
    SELECT m AS month, charged, paid,
           v_saldo + SUM(charged - paid) OVER (ORDER BY m) AS saldo_out
    FROM grid
    ORDER BY m;


    SELECT COALESCE(SUM(amount), 0) INTO v_charged
    FROM charges WHERE account_id = p_account_id AND YEAR(period) = p_year;


    SELECT COALESCE(SUM(amount), 0) INTO v_paid
    FROM payments WHERE account_id = p_account_id AND YEAR(pay_date) = p_year;


    -- начисление последнего месяца с начислениями (текущий счёт к оплате)
    SELECT COALESCE(SUM(amount), 0) INTO v_last_charge
    FROM charges
    WHERE account_id = p_account_id
      AND period = (SELECT MAX(period) FROM charges
                    WHERE account_id = p_account_id AND YEAR(period) = p_year);


    SELECT a.account_no, a.flat, a.owner,
           v_saldo                                    AS saldo_in,
           v_charged                                  AS total_charged,
           v_paid                                     AS total_paid,
           v_saldo + v_charged - v_paid - v_last_charge AS debt_prev,
           v_saldo + v_charged - v_paid               AS to_pay
    FROM accounts a
    WHERE a.id = p_account_id;
END$$
```

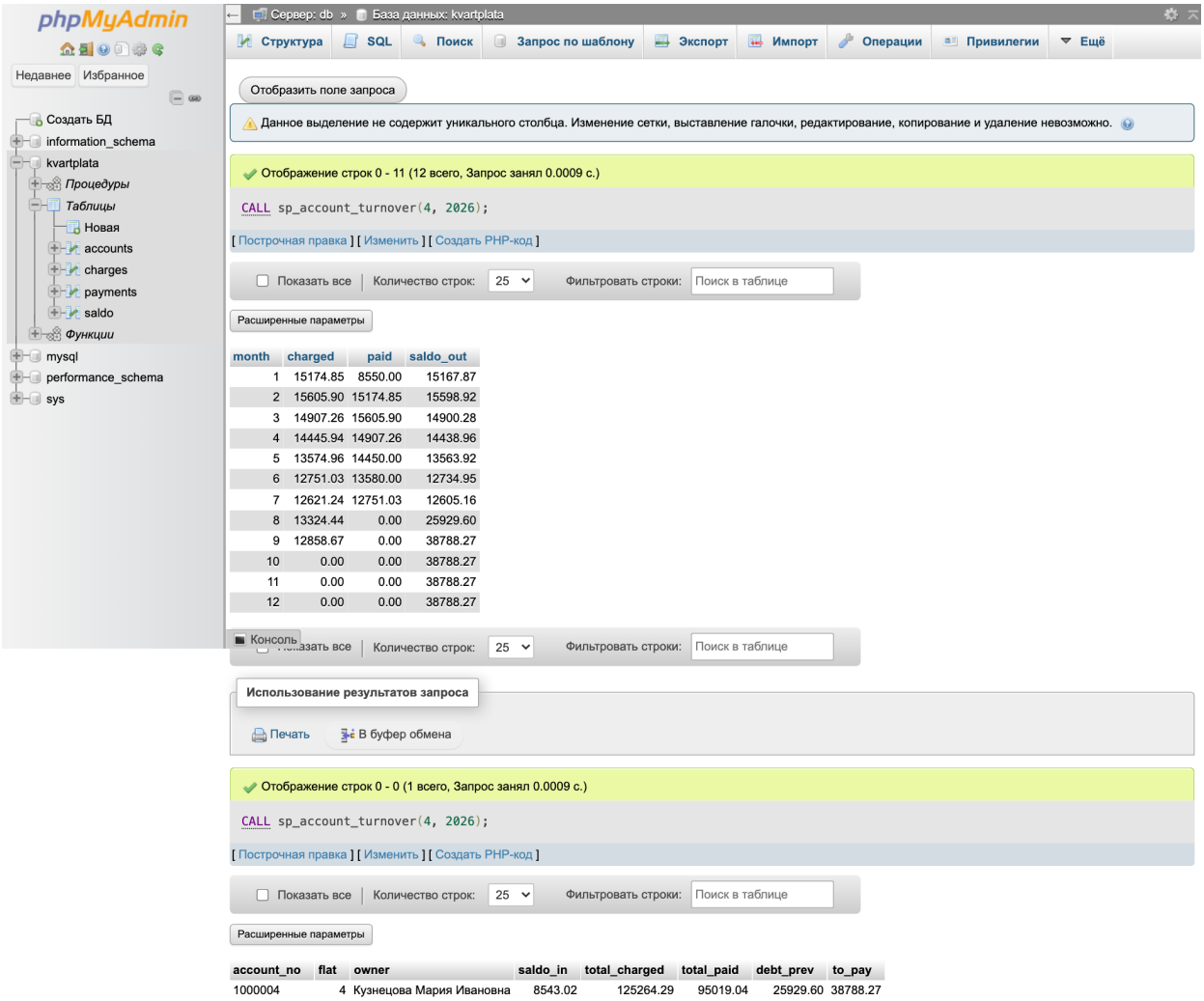


Рисунок 19 – Вызов sp_account_turnover(4, 2026): два набора результатов

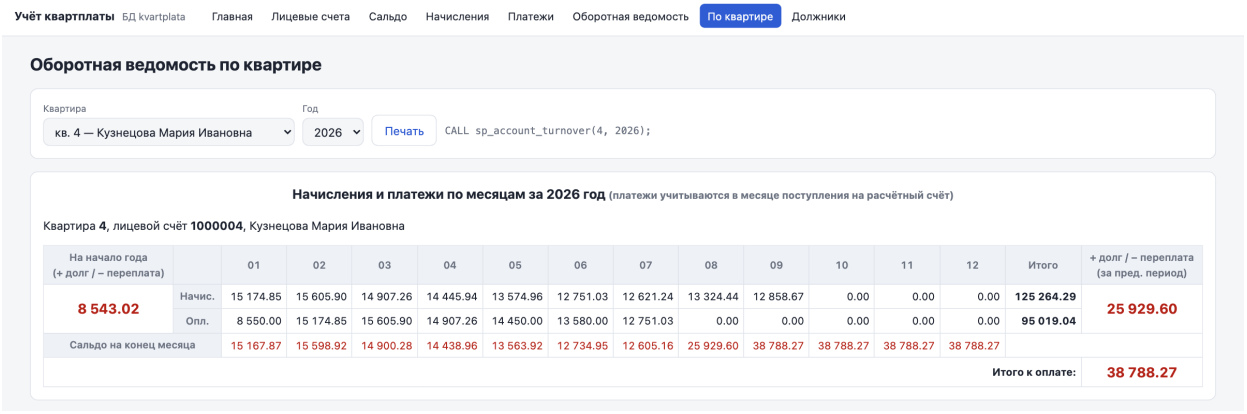


Рисунок 20 – Оборотная ведомость по квартире 4

Проверка по квартире 4: 8 543.02 + 125 264.29 − 95 019.04 = 38 788.27 (итого к оплате); долг за предыдущие периоды 38 788.27 − 12 858.67 (начислено за сентябрь) = 25 929.60.

Для квартиры 6 с переплатой вместо строки «Итого к оплате» выводится строка «Переплата», а сальдо выделено зелёным цветом (рисунок 21).

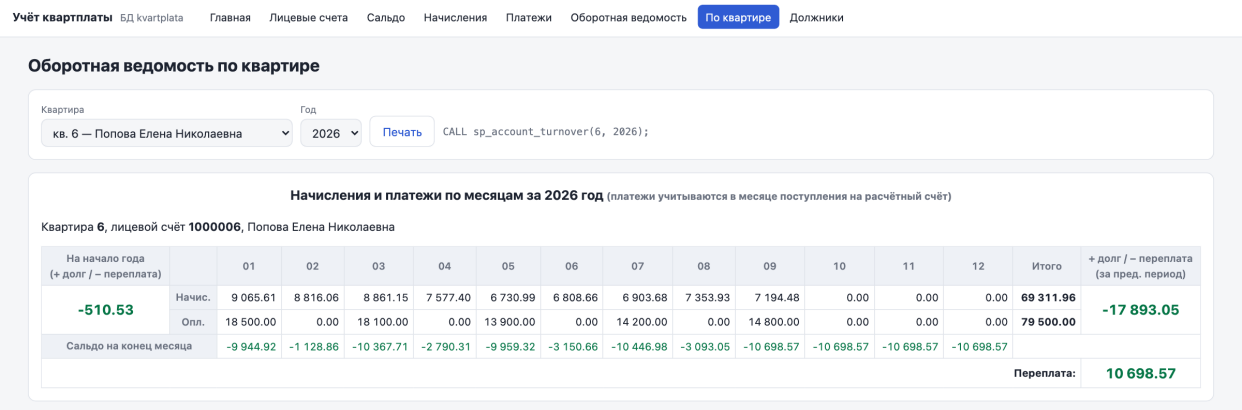


Рисунок 21 – Оборотная ведомость по квартире 6 с переплатой

3.4. Модуль 3. Сводка по категориям должников

Процедура sp_debtors выводит квартиры с положительным сальдо (долгом) на 1-е число указанного месяца: начисление за последний месяц, сальдо и сумму долга в одной из колонок «1 месяц», «2 месяца», «3 месяца», «Свыше 3 месяцев».

Категория определяется так: начисления по квартире суммируются от последнего месяца к более ранним (оконная функция SUM() OVER (ORDER BY period DESC)), и долг относится к категории N, где N – наименьшее число последних месяцев, начисления за которые покрывают сумму долга. Например, у квартиры 1 долг 19 391.39 больше начисления за сентябрь (9 962.39), но меньше суммы начислений за сентябрь и август (9 962.39 + 9 459.78 = 19 422.17), поэтому долг относится к категории «2 месяца». Это правило совпадает со строками образца из задания.

```
CREATE PROCEDURE sp_debtors(IN p_date DATE)
    COMMENT 'Должники по категориям (1, 2, 3, свыше 3 месяцев)'
BEGIN
    DECLARE v_date DATE DEFAULT DATE_FORMAT(p_date, '%Y-%m-01');

    WITH monthly AS (
        SELECT account_id, period, SUM(amount) AS amount
        FROM charges
        WHERE period < v_date
        GROUP BY account_id, period
    ),
    cum AS (
        SELECT account_id, period, amount,
               SUM(amount) OVER (PARTITION BY account_id ORDER BY period DESC) AS total
        FROM monthly
    ),
    debt AS (
        SELECT a.id, a.flat, a.account_no, a.owner,
               fn_saldo(a.id, v_date) AS saldo,
               (SELECT COALESCE(SUM(amount), 0) FROM monthly
                 WHERE account_id = a.id
```

```
                    AND period = DATE_SUB(v_date, INTERVAL 1 MONTH)) AS last_charge
        FROM accounts a
    ),
    graded AS (
        SELECT d.*,
               1 + (SELECT COUNT(*) FROM cum
                    WHERE cum.account_id = d.id AND cum.total < d.saldo) AS months
        FROM debt d
        WHERE d.saldo > 0
    )
    SELECT flat, account_no, owner, last_charge, saldo, months,
           IF(months = 1, saldo, NULL) AS debt_1,
           IF(months = 2, saldo, NULL) AS debt_2,
           IF(months = 3, saldo, NULL) AS debt_3,
           IF(months > 3, saldo, NULL) AS debt_over_3
    FROM graded
    ORDER BY flat;
END$$
```

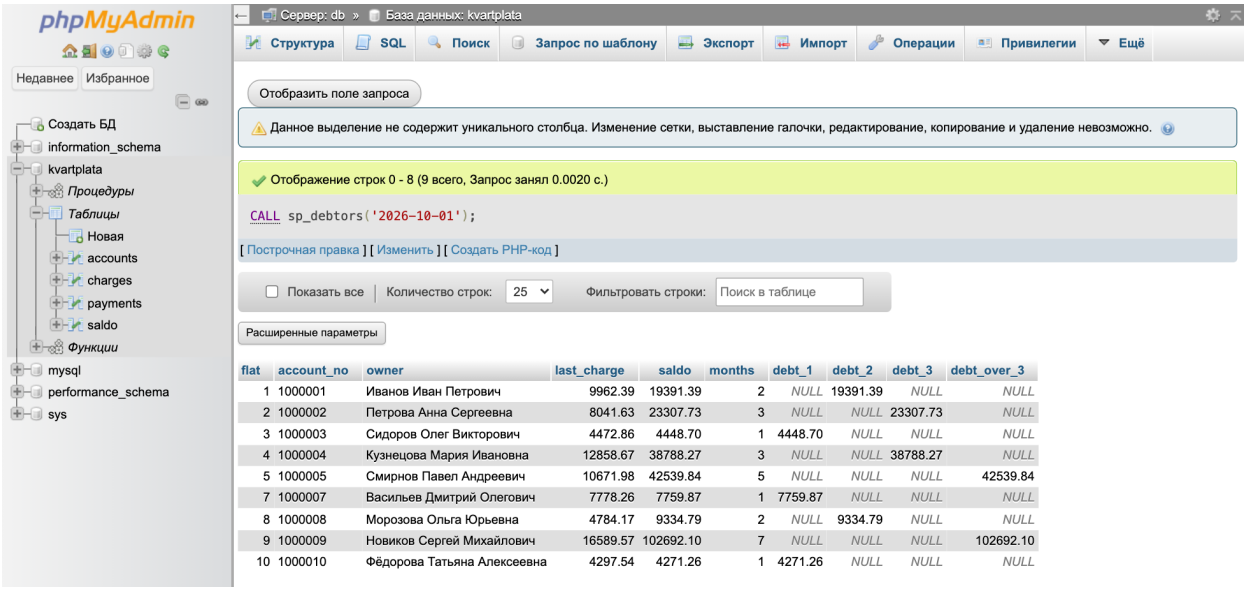


Рисунок 22 – Вызов sp_debtors('2026-10-01') в phpMyAdmin

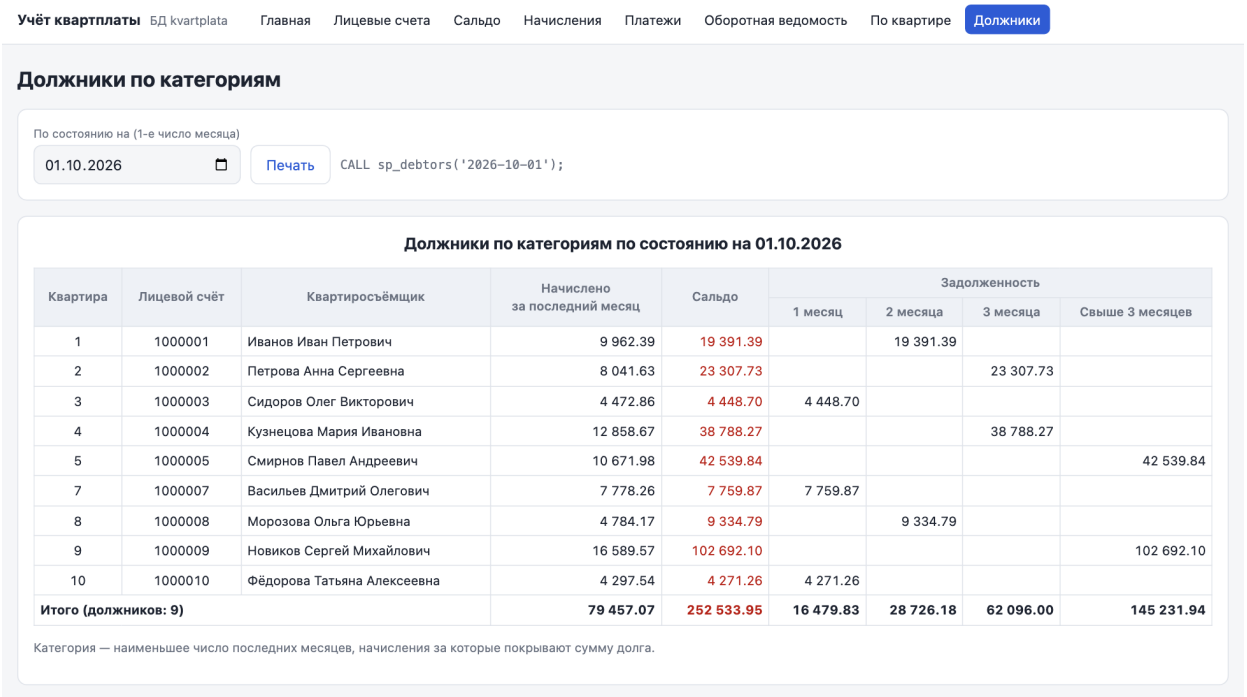


Рисунок 23 – Сводка по категориям должников на 01.10.2026 в веб-интерфейсе

Квартира 6 в сводку не попала, так как у неё переплата. Отчёт можно построить на любую дату: на рисунке 24 показано состояние на 01.07.2026.

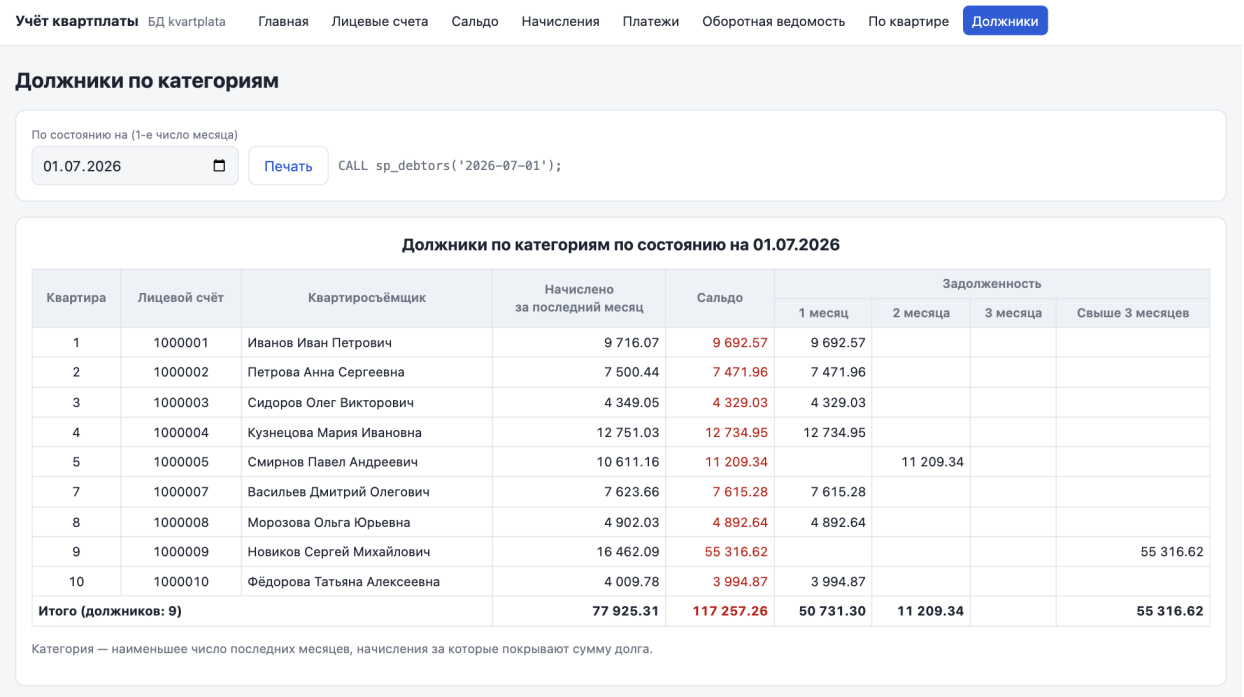


Рисунок 24 – Сводка по категориям должников на 01.07.2026

3.5. Закрытие месяца

Процедура sp_close_month записывает в таблицу saldo рассчитанное сальдо всех лицевых счетов на начало следующего месяца. После этого расчёт сальдо функцией fn_saldo начинается с записанного значения. Повторный вызов за тот же месяц пересчитывает записанные ранее значения.

```
CREATE PROCEDURE sp_close_month(IN p_period DATE)
    COMMENT 'Записать в saldo сальдо на начало следующего месяца'
BEGIN
    DECLARE v_next DATE DEFAULT DATE_ADD(DATE_FORMAT(p_period, '%Y-%m-01'), INTERVAL 1
MONTH);


    -- старая запись мешала бы расчёту: fn_saldo взял бы её как отправную точку
    DELETE FROM saldo WHERE period = v_next;


    DROP TEMPORARY TABLE IF EXISTS tmp_saldo;
    CREATE TEMPORARY TABLE tmp_saldo AS
        SELECT id AS account_id, fn_saldo(id, v_next) AS amount FROM accounts;


    INSERT INTO saldo (account_id, period, amount)
    SELECT account_id, v_next, amount FROM tmp_saldo;


    SELECT COUNT(*) AS accounts, v_next AS period FROM tmp_saldo;
    DROP TEMPORARY TABLE tmp_saldo;
END$$
```

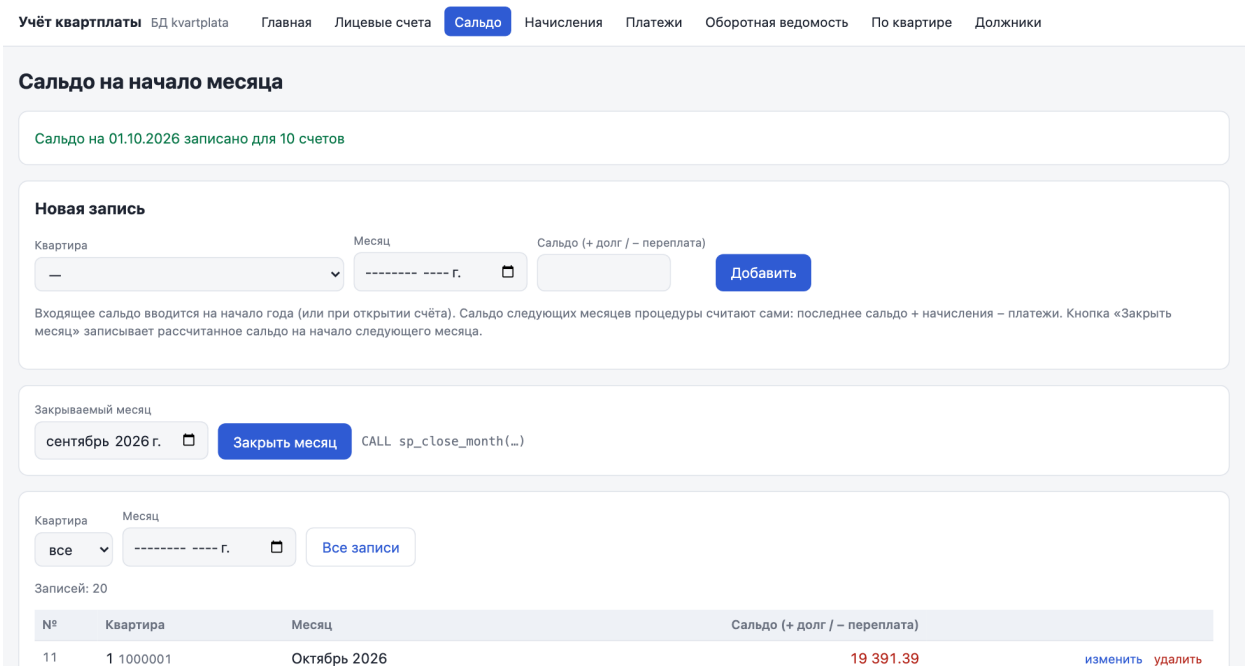


Рисунок 25 – Результат закрытия сентября 2026 года: записано сальдо на 01.10.2026

3.6. Проверка работоспособности

Правильность расчётов проверена сравнением сальдо, вычисленного функцией fn_saldo, с ручным расчётом «сальдо на 01.01 + все начисления − все платежи» по каждой квартире. Значения совпали для всех 10 квартир (рисунок 26).

```
SELECT a.flat,
       fn_saldo(a.id, '2026-01-01')
         + (SELECT COALESCE(SUM(amount), 0) FROM charges
            WHERE account_id = a.id AND period < '2026-10-01')
         - (SELECT COALESCE(SUM(amount), 0) FROM payments
            WHERE account_id = a.id AND pay_date < '2026-10-01') AS by_hand,
       fn_saldo(a.id, '2026-10-01') AS by_function
FROM accounts a ORDER BY a.flat;
```

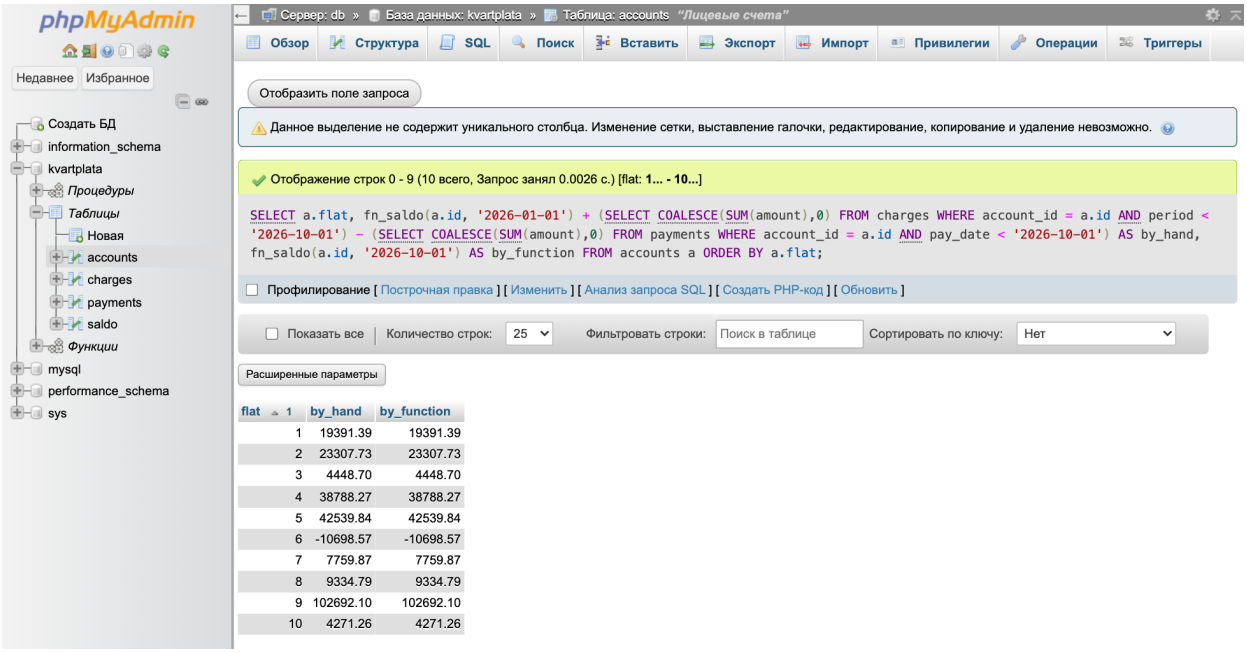


Рисунок 26 – Сверка сальдо: ручной расчёт и функция fn_saldo

Также проверено, что отчёты сразу отражают изменения в таблицах. Через веб-интерфейс введён платёж по квартире 4 на сумму 25 929.60 от 20.09.2026 (рисунок 27).

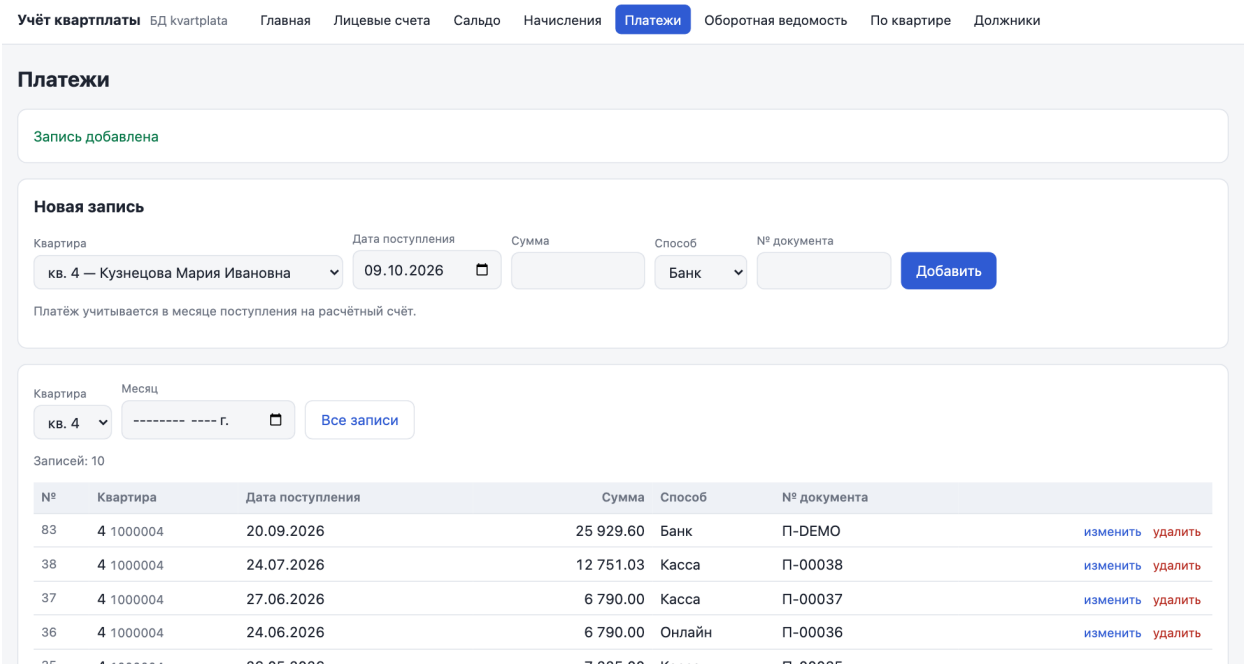


Рисунок 27 – Добавление платежа по квартире 4

После этого в оборотной ведомости по квартире 4 в сентябре появилась оплата, сумма к оплате уменьшилась до 12 858.67 (начисление за сентябрь), а в сводке должников квартира 4 перешла из категории «3 месяца» в категорию «1 месяц» (рисунки 28 и 29).

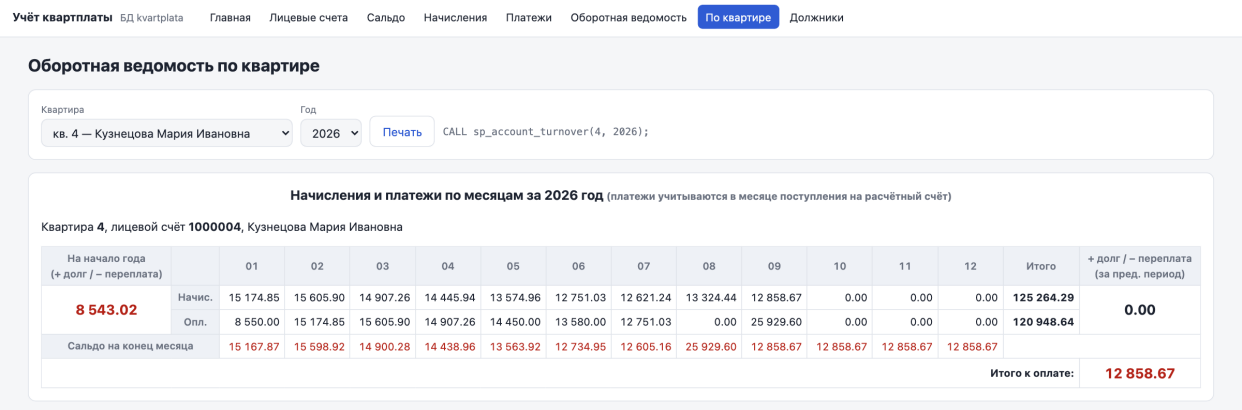


Рисунок 28 – Оборотная ведомость по квартире 4 после ввода платежа

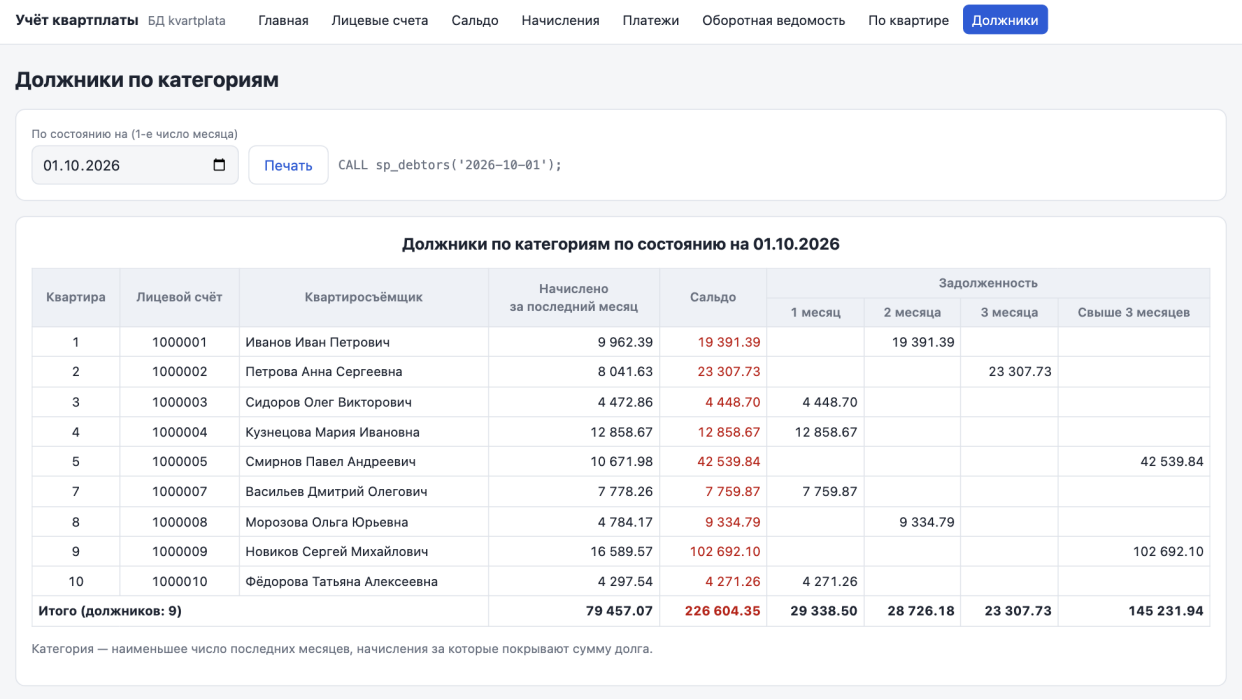


Рисунок 29 – Сводка по категориям должников после ввода платежа

Выводы

В ходе лабораторной работы:

- создана база данных kvartplata с таблицами saldo, charges, payments и справочником лицевых счетов accounts; целостность данных обеспечена внешними ключами, ограничениями UNIQUE и CHECK;
- таблицы заполнены тестовыми данными по 10 квартирам за январь–сентябрь 2026 года;
- разработан веб-интерфейс на PHP для ввода, изменения и удаления записей во всех таблицах с проверкой данных и выводом понятных сообщений об ошибках;
- реализованы три модуля вывода информации на основе хранимых процедур: оборотная ведомость (sp_turnover_sheet), оборотная ведомость по квартире (sp_account_turnover) и сводка по категориям должников (sp_debtors), а также функция расчёта сальдо fn_saldo и процедура закрытия месяца sp_close_month;
- работоспособность процедур проверена в phpMyAdmin и через веб-интерфейс, результаты сверены с ручным расчётом.

Перенос расчётов в хранимые процедуры позволяет выполнять обработку данных на сервере СУБД, передавать клиенту только готовый результат и использовать одни и те же отчёты из разных приложений. Средства MySQL 8 – общие табличные выражения, рекурсивные запросы и оконные функции – позволяют вычислять накопительные итоги (сальдо на конец каждого месяца) одним запросом, без курсоров и циклов.

Приложение А

Текст хранимых процедур (файл sql/03_procedures.sql)

```
-- Пункт 4. Модули вывода информации на основе хранимых процедур
--   sp_turnover_sheet      — оборотная ведомость за год по всем квартирам
--   sp_account_turnover    — оборотная ведомость по одной квартире
--   sp_debtors             — сводка по категориям должников
-- Вспомогательные:
--   fn_saldo               — сальдо лицевого счёта на заданную дату
--   sp_close_month         — закрытие месяца (запись сальдо на начало следующего)

SET NAMES utf8mb4;
USE kvartplata;

DROP FUNCTION  IF EXISTS fn_saldo;
DROP PROCEDURE IF EXISTS sp_turnover_sheet;
DROP PROCEDURE IF EXISTS sp_account_turnover;
DROP PROCEDURE IF EXISTS sp_debtors;
DROP PROCEDURE IF EXISTS sp_close_month;

DELIMITER $$

-- Сальдо на начало дня p_date: последнее зафиксированное в saldo значение
-- плюс начисления и минус платежи после него и до p_date
CREATE FUNCTION fn_saldo(p_account_id INT UNSIGNED, p_date DATE)
    RETURNS DECIMAL(12,2)
    READS SQL DATA
    COMMENT 'Сальдо лицевого счёта на начало даты'
BEGIN
    DECLARE v_from   DATE DEFAULT '1000-01-01';
    DECLARE v_amount DECIMAL(12,2) DEFAULT 0;
    DECLARE v_charged, v_paid DECIMAL(12,2);

    SELECT period, amount INTO v_from, v_amount
    FROM saldo
    WHERE account_id = p_account_id AND period <= p_date
    ORDER BY period DESC
    LIMIT 1;

    SELECT COALESCE(SUM(amount), 0) INTO v_charged
    FROM charges
    WHERE account_id = p_account_id AND period >= v_from AND period < p_date;

    SELECT COALESCE(SUM(amount), 0) INTO v_paid
    FROM payments
    WHERE account_id = p_account_id AND pay_date >= v_from AND pay_date < p_date;

    RETURN v_amount + v_charged - v_paid;
END$$

-- Оборотная ведомость за год.
-- Строка = квартира × месяц: начислено, оплачено, сальдо на конец месяца.
-- Месяцы после последнего месяца с движением в этом году не выводятся.
CREATE PROCEDURE sp_turnover_sheet(IN p_year SMALLINT)
    COMMENT 'Оборотная ведомость за год по всем квартирам'
BEGIN
    DECLARE v_start DATE DEFAULT MAKEDATE(p_year, 1);
    DECLARE v_last  TINYINT;

    SELECT MAX(m) INTO v_last FROM (
        SELECT MONTH(period) AS m FROM charges WHERE YEAR(period) = p_year
        UNION ALL
        SELECT MONTH(pay_date) FROM payments WHERE YEAR(pay_date) = p_year
    ) t;
```

```
    WITH RECURSIVE months AS (
        SELECT 1 AS m
        UNION ALL
        SELECT m + 1 FROM months WHERE m < COALESCE(v_last, 0)
    ),
    ch AS (
        SELECT account_id, MONTH(period) AS m, SUM(amount) AS amount
        FROM charges WHERE YEAR(period) = p_year
        GROUP BY account_id, MONTH(period)
    ),
    pay AS (
        SELECT account_id, MONTH(pay_date) AS m, SUM(amount) AS amount
        FROM payments WHERE YEAR(pay_date) = p_year
        GROUP BY account_id, MONTH(pay_date)
    ),
    grid AS (
        SELECT a.id AS account_id, a.flat, fn_saldo(a.id, v_start) AS saldo_in,
               mo.m, COALESCE(ch.amount, 0) AS charged, COALESCE(pay.amount, 0) AS paid
        FROM accounts a
        CROSS JOIN months mo
        LEFT JOIN ch  ON ch.account_id  = a.id AND ch.m  = mo.m
        LEFT JOIN pay ON pay.account_id = a.id AND pay.m = mo.m
    )
    SELECT account_id, flat, saldo_in, m AS month, charged, paid,
           saldo_in + SUM(charged - paid) OVER (PARTITION BY account_id ORDER BY m) AS
saldo_out
    FROM grid
    ORDER BY flat, m;
END$$


-- Оборотная ведомость по квартире за год. Два набора результатов:
--   1) по месяцам: начислено, оплачено, сальдо на конец месяца;
--   2) итоги: сальдо на начало года, всего начислено/оплачено,
--      долг (+) / переплата (−) за предыдущие периоды, итого к оплате.
CREATE PROCEDURE sp_account_turnover(IN p_account_id INT UNSIGNED, IN p_year SMALLINT)
    COMMENT 'Начисления и платежи по месяцам для одной квартиры'
BEGIN
    DECLARE v_start   DATE DEFAULT MAKEDATE(p_year, 1);
    DECLARE v_saldo   DECIMAL(12,2) DEFAULT fn_saldo(p_account_id, v_start);
    DECLARE v_charged, v_paid, v_last_charge DECIMAL(12,2);


    WITH RECURSIVE months AS (
        SELECT 1 AS m UNION ALL SELECT m + 1 FROM months WHERE m < 12
    ),
    grid AS (
        SELECT mo.m,
               (SELECT COALESCE(SUM(amount), 0) FROM charges
                 WHERE account_id = p_account_id
                   AND period = DATE_ADD(v_start, INTERVAL mo.m - 1 MONTH)) AS charged,
               (SELECT COALESCE(SUM(amount), 0) FROM payments
                 WHERE account_id = p_account_id
                   AND YEAR(pay_date) = p_year AND MONTH(pay_date) = mo.m) AS paid
        FROM months mo
    )
    SELECT m AS month, charged, paid,
           v_saldo + SUM(charged - paid) OVER (ORDER BY m) AS saldo_out
    FROM grid
    ORDER BY m;


    SELECT COALESCE(SUM(amount), 0) INTO v_charged
    FROM charges WHERE account_id = p_account_id AND YEAR(period) = p_year;


    SELECT COALESCE(SUM(amount), 0) INTO v_paid
    FROM payments WHERE account_id = p_account_id AND YEAR(pay_date) = p_year;


    -- начисление последнего месяца с начислениями (текущий счёт к оплате)
    SELECT COALESCE(SUM(amount), 0) INTO v_last_charge
    FROM charges
    WHERE account_id = p_account_id
      AND period = (SELECT MAX(period) FROM charges
```

```
                    WHERE account_id = p_account_id AND YEAR(period) = p_year);

    SELECT a.account_no, a.flat, a.owner,
           v_saldo                                    AS saldo_in,
           v_charged                                  AS total_charged,
           v_paid                                     AS total_paid,
           v_saldo + v_charged - v_paid - v_last_charge AS debt_prev,
           v_saldo + v_charged - v_paid               AS to_pay
    FROM accounts a
    WHERE a.id = p_account_id;
END$$


-- Сводка по категориям должников на дату p_date (берётся 1-е число месяца).
-- Задолженность относится к категории «N месяцев», где N — наименьшее число
-- последних месяцев, начисления за которые покрывают сумму долга.
CREATE PROCEDURE sp_debtors(IN p_date DATE)
    COMMENT 'Должники по категориям (1, 2, 3, свыше 3 месяцев)'
BEGIN
    DECLARE v_date DATE DEFAULT DATE_FORMAT(p_date, '%Y-%m-01');

    WITH monthly AS (
        SELECT account_id, period, SUM(amount) AS amount
        FROM charges
        WHERE period < v_date
        GROUP BY account_id, period
    ),
    cum AS (
        SELECT account_id, period, amount,
               SUM(amount) OVER (PARTITION BY account_id ORDER BY period DESC) AS total
        FROM monthly
    ),
    debt AS (
        SELECT a.id, a.flat, a.account_no, a.owner,
               fn_saldo(a.id, v_date) AS saldo,
               (SELECT COALESCE(SUM(amount), 0) FROM monthly
                 WHERE account_id = a.id
                   AND period = DATE_SUB(v_date, INTERVAL 1 MONTH)) AS last_charge
        FROM accounts a
    ),
    graded AS (
        SELECT d.*,
               1 + (SELECT COUNT(*) FROM cum
                      WHERE cum.account_id = d.id AND cum.total < d.saldo) AS months
        FROM debt d
        WHERE d.saldo > 0
    )
    SELECT flat, account_no, owner, last_charge, saldo, months,
           IF(months = 1, saldo, NULL) AS debt_1,
           IF(months = 2, saldo, NULL) AS debt_2,
           IF(months = 3, saldo, NULL) AS debt_3,
           IF(months > 3, saldo, NULL) AS debt_over_3
    FROM graded
    ORDER BY flat;
END$$


-- Закрытие месяца: фиксирует сальдо на начало месяца, следующего за p_period.
-- Повторный вызов пересчитывает ранее записанные значения.
CREATE PROCEDURE sp_close_month(IN p_period DATE)
    COMMENT 'Записать в saldo сальдо на начало следующего месяца'
BEGIN
    DECLARE v_next DATE DEFAULT DATE_ADD(DATE_FORMAT(p_period, '%Y-%m-01'), INTERVAL 1
MONTH);

    -- старая запись мешала бы расчёту: fn_saldo взял бы её как отправную точку
    DELETE FROM saldo WHERE period = v_next;

    DROP TEMPORARY TABLE IF EXISTS tmp_saldo;
    CREATE TEMPORARY TABLE tmp_saldo AS
        SELECT id AS account_id, fn_saldo(id, v_next) AS amount FROM accounts;
```

```
    INSERT INTO saldo (account_id, period, amount)
    SELECT account_id, v_next, amount FROM tmp_saldo;


    SELECT COUNT(*) AS accounts, v_next AS period FROM tmp_saldo;
    DROP TEMPORARY TABLE tmp_saldo;
END$$


DELIMITER ;
```